

ElectionGuard: Formalization and Proof of Threshold Privacy

Abstract—ElectionGuard is a versatile toolkit used by Microsoft to provide private and end-to-end verifiable elections. It has been successfully trialed in U.S. elections (e.g., Wisconsin, Idaho, Maryland, 2020 U.S. Congress). While it is an important system used in real-world elections, its privacy has been minimally explored and relied on incomplete security statements (e.g., Benaloh et al. USENIX 2024).

In this paper, we first formalize ElectionGuard algorithmically - including the distributed key generation and threshold decryption. Then, we address an incorrectly claimed result regarding their threshold encryption scheme (a variant of ElGamal), identify and reference all cryptographic building blocks, and show a vulnerability for distributed key generation. Moreover, we introduce the novel concept of *multi-prover zero-knowledge for a linear relation* and show it can be instantiated with the actual proof that ElectionGuard uses. Finally, we provide a proof of threshold privacy for its current two variants - v2.0 and v2.1.

I. INTRODUCTION

Electronic voting (e-voting) systems are being deployed in real-world elections, e.g., Switzerland, Estonia, USA. Most elections rely on *in-person e-voting* where voters cast their vote (as a ballot) in a secure polling station, and thus ensuring *vote privacy* and *verifiability*. Privacy is provided by the voter having complete control over their ballot - receiving a blank ballot, selecting their choice, and anonymously casting it. *End-to-end verifiability* [1] looks at all aspects of the voting process are verified, including when an individual voter can check that their own ballot is correctly cast, recorded and tallied.

Some elections consider *online e-voting* where the voter can create and cast their votes from anywhere, especially from outside of the supervised environment of a polling station. In this context, *privacy* is achieved by encrypting the voter's choice, and *end-to-end verifiability* relies on individual voters checking that their ballot encrypts the right vote (e.g., using Benaloh challenge [1]) and that is recorded correctly in the ballot box, together with publicly verifiable election result. Often e-voting systems (including the main focus of this paper - ElectionGuard) provide the result using homomorphic encryption and as such the individual verification can only be done over encrypted votes. Alternative methods (e.g., in Selene [2]) consider tracker numbers and mixnets for voters to individually check their plain vote in the election result.

Both methods of e-voting consider *threshold* mechanism to manage trust in election officials. Election officials become *guardians* and produce the election result only when the *quorum* is met - the number of guardians involved are greater than the predetermined threshold.

ElectionGuard [3], [4] is an e-voting toolkit introduced by Microsoft in 2019 to provide *private* and *end-to-end verifiable*

elections. What makes ElectionGuard different from many of the academic approaches to e-voting is the versatility of the toolkit that allows for many customization: in-person or online or by mail, electronic ballots or paper ballots, voting or auditing, direct-electronic recording devices or optical scan counting software. It has been trialed successfully in some public U.S. elections (e.g., in Wisconsin, California, Idaho, Utah, Maryland) and one U.S. Congress election (during the 2020 pandemic). Currently, there are two recommended variants: EG-v2.0 [3] and EG-v2.1 [4], [5], and four deprecated versions: v1.1, v1.0, v0.95, and v0.85¹. EG-v2.0 and EG-v2.1 rely on the same core primitives with subtle differences.

While ElectionGuard is such an important toolkit and used in real-world elections, there are very few results that look in-depth at its security guarantees and underlying cryptographic primitives - especially, when they rely on a bespoke model for the distributed key generation (DKG), for some of the zero-knowledge proofs, and for the encryption.

Benaloh et al. [6] and Chen-Lindell [7] show that ElectionGuard's distributed key generation holds even for an dishonest majority - under standard communication channel assumptions (e.g., confidential and authenticated). Then, [6] makes two claims (that align with the specifications of EG-v2.0 and EG-v2.1): there is no PKI infrastructure and there is no need for authentication. This clearly weakens the security guarantees, but they believe they can enforce an honest-but-curious attacker model for the guardians in DKG, as any errors (or malicious inconsistencies) will be detected and addressed out-of-band. The EG-v2.0 specifications aim to publicly address any issues in DKG and potentially identify the malicious party, without clear details on how this can be achieved.

We will show that the lack of authentication is a significant issue, as guardians can be malicious in offline distributed key generation, and external attackers can easily impersonate guardians in an online distributed key generation. To address this it will require parties signing messages (significant update to the specifications) and updating the source-code just before election starts; aspect, that it's hard to do on the fly and without proper validation.

Haines et al. [8] provide a formal verification of the verifiability ElectionGuard v0.85 in a symbolic prover - Coq, where they assume the distributed key generation is honestly produced and focuses on guardians being malicious in the tally

¹<https://www.electionguard.vote/spec/>

phase. This version of ElectionGuard uses a simple model of guardians producing individual Schnorr proofs for tally correctness - this is different from the multi-prover model in EG-v2.0 and EG-v2.1 where all provers collaborate together to produce the proof of tally correctness.

Contributions: In this paper we propose an accurate *cryptographic model* for ElectionGuard together with the *first formal proof of privacy* for its two current variants: EG-v2.0 and EG-v2.1. This is a significant endeavor due to the large system specification documentation for EG-v2.0 and EG-v2.1 (with subtle differences between them), most cryptographic primitives used are not referenced, and the main encryption scheme contains incorrect security claims. More precisely, we have achieved the following:

- *an accurate cryptographic model of ElectionGuard.* We identify all building blocks and their existing security guarantees, and when needed complete some missing security claims.
- *address incorrect security claims and show a vulnerability.* We prove a series of security results for the main encryption scheme and address an incorrect claim in [5] regarding its threshold security. Additionally, we show that *lack of authentication* can be a significant issue and their attacker model (i.e. honest-but-curious) is not sufficient. This can lead to insecure distributed key generation process when done in an online manner (that is allowed by the toolkit).
- *design a novel multi-prover zero-knowledge proof.* We introduce an efficient and elegant mechanism to prove tally correctness, that is compatible with the specific multi-prover proof in EG-v2.0 and EG-v2.1. We prove that our construction satisfies standard properties (e.g., completeness, proof of knowledge, and zero-knowledge) and *traitor tracing* where it identifies if any of the provers misbehaves.
- *a definition of threshold privacy, and a proof of it for ElectionGuard.* The previous work of Benaloh et al. [5] has only informally treated the privacy of ElectionGuard, and in this paper we provide rigorous proofs of threshold privacy. Cortier et al. [9] produced a proof for an earlier version, ElectionGuard v.0.85, due to similarities with Helios. Additionally, we introduce an improved and complete version of threshold privacy based on Cortier et al. [9] and [10] - we highlight that [9] considers a weak model of threshold corruption.

II. PRELIMINARIES

We introduce the core building blocks and assumptions needed to describe and analyze ElectionGuard.

A. Cryptographic basic primitives

Cyclic Group: We consider the cyclic group (p, q, g) defined by the generator g and primes p and q . As q is the order of the group the following property holds $p = r \cdot q + 1$ for some integer $r \in \mathbb{Z}_+^*$. For ease of expression, we introduce the algorithm $(p, q, g) \leftarrow \text{GSetup}(\lambda)$.

Hash Functions: We consider the hash functions as $H : \{0, 1\}^{256} \times \{0, 1\}^* \rightarrow \{0, 1\}^{256}$. Often, the output has to be in a specific group $\mathbb{Z}_q$ and this is represented as $H(x; y) \bmod q$. Due to the choice of q being extremely close to output space 2^{256} , we simply drop the modulo and always use $H(x; y)$. Hash functions are *collision resistant* if no efficient adversary $\mathcal{A}$ can produce two distinct values y_1, y_2 such that $H(x, y_1) = H(x, y_2)$, for some x . In most security proofs, hash functions are abstracted as *random-oracles* - ideal tables that keep track of input-output pairs. ElectionGuard implements the hash function H using HMAC-SHA-256 [11].

Commitment Scheme: We consider the deterministic commitment schemes $\text{com} \leftarrow \text{Com}([\text{aux}], r)$ for any random value $r \leftarrow \{0, 1\}^\lambda$ and some public auxiliary input $[\text{aux}]$. Verification is enabled by re-running the same commitment algorithm. Commitment schemes satisfy two main properties: *binding* - only one valid commitment value per input, and *hiding* - leaks negligible information about the input. ElectionGuard uses $\text{HCom}([\text{aux}], r) = H(\text{aux}, r)$ [12] based on a hash function H , that has shown to satisfy hiding and binding properties.

B. Non-Interactive Proof Systems

We introduce non-interactive zero-knowledge proof systems $\text{PS} = (R, P, V)$ that consists of

- $\{0, 1\} \leftarrow R(w, \text{st})$ a deterministic relation over a *statement* st and *witness* w . Sometimes, the statement may contain public parameters (e.g., the cyclic group used). Often, the relation is NP, and Athamnah et al. [13] have shown that this can be evaluated in polynomial time.
- $\pi \leftarrow P(w : \text{st}, [\text{aux}])$ is a probabilistic algorithm that produced a proof π for a statement st , given access to the witness w . This is transform into an *signature of knowledge* [14] by including an auxiliary message $[\text{aux}]$, thus obtaining some unforgeability for the auxiliary message.
- $\{0, 1\} \leftarrow V(\text{st}, \pi, [\text{aux}])$ is a deterministic algorithm that verifies the proof π and statement st . For any valid relation $R(w, \text{st})$, *completeness* ensures with overwhelming probability that $1 \leftarrow V(\text{st}, P(w : \text{st}, [\text{aux}]), [\text{aux}])$.

The proof systems we consider are in the Random Oracle Model (ROM), that we can abstract away based on Nguyen et al. [15] where they show that any result that was obtained using a blackbox reduction without ROM also holds if the primitives and adversary are given ROM access.

We consider the *zero-knowledge property* (ZKP) [16] that ensures the proof leaks no information beyond the fact that the relation holds. Formally, an efficient adversary $\mathcal{A}$ has to distinguish between $\pi \leftarrow P(w : \text{st}, [\text{aux}])$ and $\pi \leftarrow \text{Sim}(\text{st}, [\text{aux}])$ for some simulator Sim and $1 \leftarrow R(w, \text{st})$.

To ensure that honest verifiers are not convinced to accept false statements, proof systems rely on *soundness* or the stronger variant - *proof of knowledge* (PoK). In a proof of knowledge, there exists an extractor Ext that will always produce witnesses for any (adversarial produced) statement and proof that an honest verifier accepts. When combining

Share(pp, s, [aux])	Vf(pp, pub, (j, sh _j), [aux])	Rec(pp, J, (sh _j) _{j∈J} , [aux])
1: ((p, q, g), t, n, hkey) ← pp 2: a ₀ ← s 3: A ₀ ← g ^{a₀} 4: for j ∈ {1, ..., t-1} do 5: a _j ← \$Z _q 6: A _j ← g ^{a_j} mod p 7: π _j ← SchZKP.P(a _j : pp, A _j , [aux, j]) 8: P(x) = a _{t-1} x ^{t-1} + ... + a ₁ x + a ₀ 9: for ℓ ∈ {1, ..., n} do 10: sh _ℓ ← P(ℓ) 11: π ← (π ₀ , ..., π _{t-1}) 12: pub ← (A ₀ , ..., A _{t-1} , π) 13: return ((sh ₁ , ..., sh _n), pub)	1: ((p, q, g), t, n, hkey) ← pp 2: (A ₀ , ..., A _{t-1} , π) ← pub 3: S _j ← ∏ _{ℓ=0} ^t A _ℓ ^{j_ℓ} mod p 4: e _t ← g ^{sh_j} = S _j 5: e ← PubVf(pp, pub, [aux]) 6: return e ∧ e _t PubVf(pp, pub, [aux]) 1: ((p, q, g), t, n, hkey) ← pp 2: (A ₀ , ..., A _{t-1} , π) ← pub 3: (π ₀ , ..., π _{t-1}) ← π 4: for j ∈ {0, ..., t-1} do 5: e _j ← SchZKP.Vf(pp, A _j , π _j , [aux, j]) 6: e ← e ₀ ∧ ... ∧ e _{t-1} 7: return e	1: ((p, q, g), t, n, hkey) ← pp 2: if ¬PubVf(pp, pub, [aux]) return ⊥ 3: for j ∈ J do 4: L _j ← ∏ _{i∈J\{j}} (i/(i-j)) mod q 5: s ← ∑ _{j∈J} sh _j · L _j mod q 6: return s RecE(pp, J, (SH _j) _{j∈J} , [aux]) 1: ((p, q, g), t, n, hkey) ← pp 2: if ¬PubVf(pp, pub, [aux]) return ⊥ 3: for j ∈ J do 4: L _j ← ∏ _{i∈J\{j}} (i/(i-j)) mod q 5: S ← ∏ _{j∈J} SH _j ^{L_j} mod q 6: return (S, {L _j j ∈ J})

Fig. 1. Verifiable Secret Sharing VSS-v2.0 [7] with SchZKP = (R, P, V).

ZKP with PoK it leads to *simulation sound extractability for proofs of knowledge* (SSE-PoK) where the extractor Ext works even on simulated proofs done by Sim.

Instances of PS: We consider the following standard and known constructions (with details in Appendix C):

- SchZKP [17] for the knowledge of an exponent a without revealing it $R(a, A) = [A = g^a \text{ mod } p]$. This is the well-known Schnorr proof, and it satisfies completeness, soundness, zero-knowledge and proof of knowledge.
- ExpoZKP [7] for showing knowledge of multiple exponentiations (a₁, ..., a_m) where A_i = g^{a_i} mod p for all i ∈ {1, ..., m}. This is a generalization of Schnorr, and satisfies completeness, soundness, zero-knowledge and PoK. The relation is

$$R((a_1, \dots, a_m), (pp, A_1, \dots, A_m)) = [A_1 = g^{a_1} \text{ mod } p \wedge \dots \wedge A_m = g^{a_m} \text{ mod } p].$$

- ChPeZKP [16] for showing the validity of the encryption by demonstrating that the vote v ∈ {0, 1} and knowledge of the randomness a used in encryption. This is the known disjunctive Chaum-Pedersen system and it satisfies completeness, soundness, zero-knowledge and PoK. The relation is defined as

$$R((v, a), (pp, A, B)) = \left[\begin{array}{l} (A = g^a \wedge B = pk^{a+v}) \\ \vee (A = g^a \wedge B = pk^{a+(1-v)}) \end{array} \right].$$

C. Verifiable Secret Sharing

We introduce the *verifiable secret sharing scheme* [7], [18] $VSS(t, n) = (\text{Share}, \text{Rec}, \text{RecE}, \text{Vf})$ for $0 < t \leq n$, where

- ((sh₁, ..., sh_n), pub) ← Share(pp, s) a probabilistic sharing procedure for secret s with the public parameters pp = ((p, q, g), t, n, hkey), for (p, q, g) ← GSetup.
- {0, 1} ← Vf(pp, pub, (j, sh_j)) verifies the share sh_j.
- s_⊥ ← Rec(pp, J, (sh_j)_{j∈J}) recovers the secret s if t ≤ |J|, and ⊥ otherwise. *Correctness* ensures that any honest run

of Rec(pp, J, Share(pp, s)) recovers the secret s and each shares verifies 1 ← Vf(Share(s)), for t ≤ |J|.

- S_⊥ ← RecE(pp, J, {SH_j})_{j∈J}) recovers the secret s in the exponent S = g^s for all SH_j = g^{sh_j}. Pedersen [19] has shown for Shamir secret sharing [20] (the underlying scheme used by Feldman [21] and ElectionGuard) that

$$\text{RecE}(pp, J, \{SH_j\}_{j \in J}) = g^{\text{Rec}(pp, J, (sh_j)_{j \in J})},$$

while still maintaining the same privacy guarantees as the initial reconstruction. We abuse notation (S_⊥, {L_j | j ∈ J}) ← RecE to also output publicly available values L_j for each party (e.g., the Lagrange coefficients).

The sharing phase is optimally composed [18] of (1) the dealer distributing the shares (i.e. by sending the output of Share) and (2) the other participants verifying their shares (i.e., by calling Vf) and raising and resolving the complains. This is often modeled in the context of having a communication channel that is either encrypted and authenticated [18] or using sign-then-encrypt methodology (or it's more efficient variant signcryption) [7].

Secrecy states that if the dealer is honest, then during the sharing phase the adversary learns no information about the secret that is shared (as long as the adversary corrupts at most (t-1) parties). *Security* looks at the context of a dishonest dealer during the sharing phase that aims to convince all honest parties that they hold valid and consistent shares. Depending on size of the honest parties, one has either *honest majority* - that is the threshold satisfies 0 < t < n/2, or *dishonest majority* with 0 < t ≤ n. A special type of security is *with aborts* where the guarantee of having a valid and consistent share is for honest parties that didn't abort, and nothing can be said about the parties that aborted.

Instantiations of VSS: ElectionGuard relies on variants of Feldman's secret sharing scheme [21]:

- VSS-v2.0: Feldman with multiple SchZKP. The dealer publishes commitments to the coefficients of the poly-

nomial as g^{a_i} for all i , together with individual Schnorr Proofs $\pi_i \leftarrow \text{SchZKP.P}(a_i : \text{pp}, g^{a_i}, j)$ that it knows the exponent (i.e., the coefficient). This is formalized in Figure 1, and used in EG-v2.0.

- VSS-v2.1 [7]: Feldman with ExpoZKP. This is similar to VSS-v2.0 except the dealer publishing a single ExpoZKP proof for all coefficient, instead of individual ones. EG-v2.1 uses this variant.

Theorem 1 (VSS properties [7]).²

VSS-v2.0 and VSS-v2.1 satisfy correctness, secrecy, security with honest majority and security with dishonest majority and aborts, assuming the communication between the parties is confidential and authenticated.

Additive Homomorphic Secret Sharing [22]: ElectionGuard relies on their parties combining shares from multiple secret sharings. In essence, each party will have a combined share as the sum of all the shares that that party received $\text{sh}_j = \sum_{i=1}^n \text{sh}_{i,j}$ for $(\dots, \text{sh}_j, \dots) \leftarrow \text{Share}(\text{pp}, (\sum_{i=1}^n s_i))$ and $(\dots, \text{sh}_{i,j}, \dots) \leftarrow \text{Share}(\text{pp}, s_i)$. We abstract this as

$$\text{Share}(\text{pp}, (\sum_{i=1}^n s_i)) = \sum_{i=1}^n \text{Share}(\text{pp}, s_i).$$

It is straightforward to see that the combined secret sharing scheme maintains the same properties as the underlying sharing schemes, assuming the exchange of shares was done over a confidential and authenticated channel.

D. Public Key Encryption

We introduce the algorithms for the public-key encryption $\text{PKE} = (\text{KGen}, \text{Enc}, \text{Dec})$ with

- $(\text{pk}, \text{sk}) \leftarrow \text{KGen}(\lambda)$ takes the security parameter λ and produces a public-secret key pair.
- $c \leftarrow \text{Enc}(\text{pk}, m)$ returns the ciphertext c that encrypts the message m using the public key pk . When we need the randomness we use $c \leftarrow \text{Enc}(\text{pk}, m; r)$.
- $m_\perp \leftarrow \text{Dec}(\text{sk}, c)$ returns either a valid message m or an error symbol $\perp$. *Correctness* ensures with overwhelming probability that $m \leftarrow \text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m))$ for any $(\text{pk}, \text{sk}) \leftarrow \text{KGen}(\lambda)$.

We extend the definition to *homomorphic PKE* by including

- $c \leftarrow \text{Add}(c_1, \dots, c_n)$ where the ciphertext c is the encryption of $(\sum_{i=1}^n m_i)$ for all $c_i \leftarrow \text{Enc}(\text{pk}, m_i)$.

Security: All PKE schemes satisfy *indistinguishability under chosen plaintext attack* (IND-CPA) where the adversary is incapable of distinguishing between encryption of two messages, in the context of them choosing the messages and having no access to decryption oracle. Stronger notions of security do consider decryption oracle access for the adversary, and they are either limited to (1) only one decryption query for a list of ciphertexts - *indistinguishability under chosen-ciphertext attack with one parallel decryption query* (IND-1-CCA) [23], [24], or (2) to

many decryption queries - *indistinguishability under chosen-ciphertext attack* (IND-CCA2) [25]). Based on [25] and [23] we have the following relations between security properties:

$$\text{IND-CPA} \Leftarrow \text{IND-1-CCA} \Leftarrow \text{IND-CCA2}$$

Bernhard et al. [16] have shown that one can build non-malleable ciphertexts by combining a malleable encryption scheme PKE (i.e., IND-CPA) with a proof system ZKP that satisfies simulation sound extractable proof of knowledge (SSE-PoK). Essentially, the IND-1-CCA encryption scheme builds a ciphertext (cip, π) with $\text{cip} \leftarrow \text{PKE.Enc}$ and $\pi \leftarrow \text{ZKP.P}$ and decryption verifies π before PKE.Dec .

Theorem 2 (Bernhard et al. [16]). $\text{IND-1-CCA} = \text{IND-CPA}$ with SSE-PoK.

ElGamal [26]: This a well-known IND-CPA encryption scheme with the encryption step as $(g^r \bmod p, m \cdot \text{pk}^r \bmod p)$ and decryption as $(m \cdot \text{pk}^r) / (g^r)^{\text{sk}}$. Moreover, Schnorr-Jakobsson [27] have shown that *ElGamal-with-SchZKP* is IND-CCA2, and claimed (and not prove) that the threshold variant of *ElGamal-with-SchZKP* will remain IND-CCA2.

DPKE: Devillez et al. encryption scheme [28]: This is a more efficient variant of ElGamal where the encryption step is $(g^r \bmod p, \text{pk}^{r+m} \bmod p)$ and decryption is done as $\log_{\text{pk}}(\text{pk}^{r+m} / (g^r)^{\text{sk}})$. For completeness, we show that $\text{DPKE} = (\text{KGen}, \text{Enc}, \text{Dec}, \text{Add})$ is IND-CPA in Section IV-A, as this was not shown by original paper [28].

EPKE: ElectionGuard variant of DHIES [29]: The scheme $\text{EPKE} = (\text{KGen}, \text{Enc}, \text{Dec})$, described in Figure 3 is similar to DHIES³, that has been shown to be IND-CCA2 in [29]. It is reasonable to expect that EPKE satisfies IND-CCA2. Additionally, one can rely on the more general result from the Encrypt-then-MAC methodology [30] that says EPKE satisfies IND-CCA2 and *ciphertext integrity*, if the underlying encryption scheme is IND-CPA and the MAC is *strongly unforgeable*. We consider the following versions (some presented in Appendix D due to space constraints):

- EPKE-v2.0 [3]. This is formalized in Figure 3 and satisfies IND-CCA2 and *ciphertext integrity*.
- EPKE-v2.1.A [4]. This is a variant where the MAC has been replaced with a SchZKP proof.
- EPKE-v2.1.B [4]. This expands the previous scheme to work on two messages and SchZKP proof.

Remark 1 (Lack of authentication). The schemes EPKE-v2.0, EPKE-v2.1.A, and EPKE-v2.1.B do not provide *authentication* or *non-repudiation*, as there is no mechanism to identify who created the ciphertexts. This can be addressed by using the *Sign-then-Encrypt* methodology [31] while preserving the IND-CCA2 property.

E. DKG and Threshold PKE

A particular type of PKE are *threshold public-key encryptions* [32] $\text{TPKE} = (\text{DKG}, \text{Enc}, \text{Dec}, \text{Combine})$ where

² [7] analyses only VSS-v2.1, but it's straightforward to see that VSS-v2.1 and VSS-v2.0 are equivalent from a security perspective.

³ Notice that the difference is between $(k_0, k_1) \leftarrow H(x)$ versus $k_0 \leftarrow H(0, H(x))$ and $k_1 \leftarrow H(1, H(x))$.

$P(\{ \text{lesk}_i \mid i \in I \}) : \text{pp}, \{ (A_i, \text{lepk}_i) \mid i \in I \}, I, (A, B), [\text{aux}]$	
Party $(i, \text{lesk}_i, \text{lepk}_i, A_i)$ does the following	
I commitment phase	
1 :	$r_i \leftarrow \mathbb{Z}_q$
2 :	$U_i \leftarrow g^{r_i} \bmod p, V_i \leftarrow A^{r_i} \bmod p$
3 :	$\text{com}_i \leftarrow \text{Com}([i, A, B, A_i, I], U_i, V_i)$
4 :	$\text{PL} \leftarrow (i, \text{com}_i)$
5 :	wait and collect $\forall j \neq i. (j, \text{com}_j)$
6 :	$\text{PL} \leftarrow (i, U_i, V_i)$
7 :	wait and collect $\forall j \neq i. (j, U_j, V_j)$ and verify $\forall j \neq i. \text{com}_j = \text{Com}([j, A, B, A_j, I], U_j, V_j)$
I challenge and response phase	
8 :	$U \leftarrow \prod_{j \in I} U_j \bmod p, V \leftarrow \prod_{j \in I} V_j \bmod p$
9 :	$\text{AS} \leftarrow \prod_{j \in I} A_j \bmod p$
10 :	$c \leftarrow H(\text{hkey}; A, B, U, V, \text{lepk}, \text{AS}, [\text{aux}])$
11 :	$z_i \leftarrow (r_i - c \cdot \text{lesk}_i) \bmod q$
12 :	$\text{PL} \leftarrow (i, z_i)$
13 :	wait and collect $\forall j \neq i. (j, z_j)$
I validate responses	
14 :	$\forall j \neq i. U'_j \leftarrow g^{z_j} \cdot \text{lepk}_j^c$
15 :	$\forall j \neq i. V'_j \leftarrow A^{z_j} \cdot A_j^c$
16 :	$\forall j \neq i. \text{if } (U_j, V_j) \neq (U'_j, V'_j) \text{ then}$
17 :	$\text{evidence} \leftarrow (j, U_j, V_j, U'_j, V'_j)$
18 :	$O \leftarrow O \cup \{\text{Dispute}(i, j, \text{evidence})\}$
I the proof	
19 :	$z \leftarrow \sum_{j \in I} z_j \bmod q$
20 :	$\pi \leftarrow (c, z)$
21 :	return π

Fig. 2. Multi-Prover Zero-Knowledge Proof System MPZKP.

$\text{KGen}((p, q, g), \text{hkey})$	
1 :	$\text{sk} \leftarrow \mathbb{Z}_q^*; \text{pk} \leftarrow g^{\text{sk}} \bmod p$
2 :	return (sk, pk)
$\text{Enc}((p, q, g), \text{hkey}, \text{pk}, m, \text{aux}_1, \text{aux}_2)$	
1 :	$r \leftarrow \mathbb{Z}_q$
2 :	$c_0 \leftarrow g^r \bmod p$
3 :	$\beta \leftarrow \text{pk}^r \bmod p$
4 :	$k \leftarrow H(\text{hkey}; \text{aux}_1, \text{pk}, c_0, \beta)$
5 :	$k_0 \leftarrow H(k; 0x01, \text{aux}_2)$
6 :	$k_1 \leftarrow H(k; 0x02, \text{aux}_2)$
7 :	$c_1 \leftarrow m \oplus k_1$
8 :	$c_2 \leftarrow H(k_0, c_0, c_1)$
9 :	return $((c_0, c_1, c_2), r)$
$\text{Dec}((p, q, g), \text{hkey}, \text{sk}, (c_0, c_1, c_2), \text{aux}_1, \text{aux}_2)$	
1 :	$\beta \leftarrow c_0^{\text{sk}} \bmod p$
2 :	$k \leftarrow H(\text{hkey}; \text{aux}_1, \text{pk}, c_0, \beta)$
3 :	$k_0 \leftarrow H(k; 0x01, \text{aux}_2)$
4 :	$k_1 \leftarrow H(k; 0x02, \text{aux}_2)$
5 :	$m \leftarrow c_1 \oplus k_1$
6 :	$c'_2 \leftarrow H(k_0, c_0, c_1)$
7 :	if $c_2 \neq c'_2$ then $m \leftarrow \perp$
8 :	return m

Fig. 3. Encryption Scheme EPKE-v2.0.

the single secret key is split into secret shares during DKG and a threshold number of them are required to decrypt (see Appendix B for syntax details). *Correctness* in this case ensures with overwhelming probability that $m \leftarrow \text{Combine}(\text{pkc}, I, \{d_i\}_{i \in I})$ over a set of $t \leq |I|$ partial decryption values $d_i \leftarrow \text{Dec}(\text{sk}_i, c)$ with $i \in I$, given the ciphertext $c \leftarrow \text{Enc}(\text{pk}, m)$ and keys $(\text{pk}, \text{pkc}, \text{sk}_1, \dots, \text{sk}_n) \leftarrow \text{DKG}(\lambda, t, n)$. Threshold IND-CPA and threshold IND-1-CCA give to the adversary the ability to corrupt up to $(t-1)$ secret share keys.

ElectionGuard relies on combining multiple VSS using homomorphic additive secret sharing. We consider DKG-v2.0⁴ (see Setup in Figure 4) the method where each party runs their own VSS-v2.0 sharing procedure, then combine individual shares using the homomorphic additive secret sharing property of VSS-v2.0. Similarly, one can define DKG-v2.1.

Benaloh et al. [5] have shown the following result for ElGamal and for DKG-v2.0. However, this result has already been shown by Cortier et al. [9]. Moreover, they do not use EPKE in the transmission of shares, they rely on the fact that the channel is authenticated and confidential. Benaloh et al. claimed this result for the ElectionGuard variant of ElGamal - *but that is incorrect as we will show in Section IV-A*.

⁴This is a correctly designed DKG where communication between parties is authenticated - in reality, EG-v2.0 and EG-v2.1 do not satisfy this.

Theorem 3 (Benaloh et al. [5], Cortier et al. [9]). *ElGamal-with-DKG-v2.0 is threshold IND-CPA.*

III. MULTI-PROVER ZERO-KNOWLEDGE PROOFS

Baum et al. [33] introduces the concept of *multi-prover zero-knowledge proofs* in the context of showing the outcome of the SPDZ secret sharing [34] is done correctly. This is a very bespoke design model where they prove each party i starts with the secret value a_i and at the end of the protocol they obtain the triple (a_i, b_i, c_i) such that $c = a * b$ for $a = \sum_{i \in I} a_i$, $b = \sum_{i \in I} b_i$ and $c = \sum_{i \in I} c_i$. This type of sharing is very common in Multi-Party Computations (MPCs) and not relevant for VSS used in e-voting.

Cui et al. [35] have extended this and shown that one can apply the MPC-in-the-head paradigm to multi-prover zero-knowledge proofs in a blackbox style for any NP relation. They rely on splitting each individual prover witnesses w_i into t shares using $w_i = \sum_{j=1}^t w_{i,j}$, then running in total $(t+1) * |I|$ 2PC (MPC for case $m=2$) over those shares. Their approach is not applicable to us due to

- **high overhead cost.** It will be more efficient just to run $|I|$ individual Schnorr proofs over each witness.
- **independent witnesses.** They assume no pre-existing relation between witnesses, while in our case additional shares (in a different format) are shared between parties.

In Figure 2 we introduce our novel multi-prover zero-knowledge MPZKP where each prover has chosen witnesses $(i, \text{lesk}_i, \text{lepk}_i)$ and together they prove that they know the exponent $\text{lesk} = \sum_{i \in I} \text{lesk}_i$ for some public commitment value $\text{lepk} = \prod_{i \in I} \text{lepk}_i$. More precisely, the relation is defined as

$$R(\{(i, \text{lesk}_i) \mid i \in I\}, (\text{pp}, A, I, \{(i, A_i, \text{lepk}_i) \mid i \in I\})) \\ = (\prod_{i \in I} A_i = A^{\sum_{i \in I} \text{lesk}_i}).$$

The essence of our construction relies on computing the same challenge c and having each prover perform a Schnorr proof $z_i \leftarrow (r_i - c \cdot \text{lesk}_i)$. Then, all Schnorr proofs are aggregated to produce system proof value $z \leftarrow \sum_{i \in I} z_i$; this is equivalent to $z \leftarrow (\sum_{i \in I} r_i - c \cdot \text{lesk})$. The verification procedure is straightforward and relies on taking the proof (c, z) for the statement $(\text{pp}, A, I, \{(i, A_i, \text{lepk}_i) \mid i \in I\})$ by computing

$$\text{AS} = \prod_{i \in I} A_i \quad \text{! this is equivalent to } A^{\text{lesk}} \\ c = H(\text{hkey}; A, B, (g^z / \text{lepk}^c), (A^z / \text{AS}^c), \text{lepk}, \text{AS}, [\text{aux}]).$$

Theorem 4. *The MPZKP in Figure 2 satisfies completeness, soundness, proof of knowledge, and zero-knowledge.*

Proof. We show that our multi-prover satisfies these properties. We are in the Random Oracle Model (ROM), and as such the extractor Ext and Sim have access and can program the output of hash functions. As stated in Section II-A we do not make explicit the calls to ROM, but they do exist and often imply replacing the Setup with a SimSetup algorithm.

Proof of completeness. On the assumption that all provers and the verifier are honest and follow the protocol, we need to show that the proof is always accepted. This reduces to showing that in the verification the same value c is obtain by recomputing its inputs in the hash function:

$$(g^z / \text{lepk}^c) = \prod_{i \in I} g^{r_i} \\ (A^z / \text{AS}^c) = \prod_{i \in I} A^{r_i} \\ \text{where } z = (\sum_{i \in I} r_i) - c \cdot \text{lesk}$$

Proof of knowledge. We will show that our system is a proof of knowledge, thus also implying soundness. The adversary $\mathcal{A}$ controls a subset of the provers $\text{advl} \subseteq I$ to produce a proof for a statement not in the language. We build the extractor $w \leftarrow \text{Ext}(\text{st}, \pi, \mathcal{A}, \text{coins})$ using

- *context:* the proof of knowledge experiment has an adversary $\mathcal{A}(\text{coins})$ that produces a statement-proof pair (st, π) . The adversary wins if $1 \leftarrow V(\text{st}, \pi)$ and $\nexists w$ such that $1 \leftarrow R(w, \text{st})$. Note that Ext only knows the witnesses (i, lesk_i) from the honest parties $i \in I \setminus \text{advl}$.
- *context:* Ext can rewind the adversary, and for the same random coins it produces the same proof and statement $(\text{st}, \pi) \leftarrow \mathcal{A}(\text{coins})$. Moreover, if one of the random coins is changed (e.g., the output of a hash value) the adversary will produce the same outcome except on the changed coin.
- *step 1:* Ext runs the adversary to obtain statement-proof pair (st, π) . The adversary controlled parties $i \in \text{advl}$

have produced intermediary values $z_i \leftarrow (r_i - c \cdot \text{lesk}_i)$. The extractor rewinds the adversary and produces $c' \neq c$ as the output to the hash function, thus also obtaining $z'_i \leftarrow (r_i - c' \cdot \text{lesk}_i)$.

- *step 2:* then, it computes $\text{lesk}_i = (z'_i - z_i) / (c'_i - c_i)$ for all $i \in \text{advl}$. Together with the witnesses it already knows for context, it returns $\{(i, \text{lesk}_i) \mid i \in I\}$.

Proof of zero-knowledge. The adversary $\mathcal{A}$ controls the verifier and a subset of the provers $\text{advl} \subseteq I$. The aim is to simulate the proof for honest parties $i \in I \setminus \text{advl}$ such that the adversary can't distinguish, only for statements in the language. We will build an simulator $\pi \leftarrow \text{Sim}(\text{st})$ using

- *context:* in zero-knowledge experiment, the adversary $\mathcal{A}$ produces the statement $\text{st} \leftarrow (\text{pp}, A, I, \{(i, A_i, \text{lepk}_i) \mid i \in I\})$ and witnesses $w \leftarrow (\{(i, \text{lesk}_i) \mid i \in I\})$, and then it continues only if $R(w, \text{st}) = 1$. The simulator doesn't see the witnesses.
- *context:* it is sufficient just to simulate the proof for only one party $i \in I \setminus \text{advl}$ to produce a proof for any A_i . This is due to the linear form of the relation where one can simulate the proof for FAS then it sets $A_i = \text{FAS} / (\prod_{j \in I \setminus \{i\}} A_j)$. All values $\{A_j \mid j \in I\}$ are already defined - they are part of the statement.
- *context:* we simulate for party $i \in I \setminus \text{advl}$ where they produce a different value $\text{FA}_i \neq A^{\text{lesk}_i}$. Recall that party i produces first (U_i, V_i) , then waits for all $\{(U_j, V_j) \mid j \in I \setminus \{i\}\}$ before returning (c, z_i) .
- *step:* The simulator samples $c, z_i \leftarrow \mathbb{Z}_q$. This is possible as the value c is the output of an hash function (or ROM) that the simulator controls. Then, it computes and sends

$$U_i = (g^{z_i} / \text{lepk}_i^c) \text{ and } V_i = A^{z_i} \text{FA}_i^c$$

□

More cryptographic details around the extraction and simulation above are found in Appendix E.

Traitor tracing in MPZKP: The proof system MPZKP provides a easy mechanism to identify if any of the provers deviated from the protocol and provided an incorrect value $A_i \neq A^{\text{lesk}_i}$. This property is relevant to stronger verifiability-style definitions (like accountability) as it will detect if provers deviate from the protocol. We achieve this by interpreting the proof (c, z_i) as a version of Schnorr proof with the verification

$$U_i = g^{z_i} \cdot \text{lepk}_i^c \text{ and } V_i = A^{z_i} \cdot A_i^c.$$

This is not exactly as Schnorr proof and it may not have the same level of soundness (as c is not recomputed). This can easily be strengthened by asking only the parties that failed this verification step to produce an independent exponential proof ExpoZKP for the relation $R(\text{lesk}_i, (\text{pp}, A, \text{lepk}_i, A_i)) = [\text{lepk}_i = g^{\text{lesk}_i} \bmod p \wedge A_i = A^{\text{lesk}_i} \bmod p]$. Notice that this doesn't contradict the proof of zero-knowledge as the adversary can't program the output of the hash function as they want - to simulate for any statement.

Secret Sharing and MPZKP: One can apply MPZKP in the context where the witnesses lesk_i are not independent, and have been build using secret sharing. Consider the secret esk has been shared into secret shares $(\dots, \text{esk}_i, \dots, \text{pub}) \leftarrow \text{VSS.Share}(\text{pp}, \text{esk})$. The system then becomes a proof of knowledge of the reconstruction, if the reconstruction can be rewritten as a linear equation of the secret shares

$$\text{Rec}(\text{pp}, l, (\text{esk}_i)_{i \in I}) = \left(\sum_{i \in I} \text{esk}_i \cdot L_i, \text{ for some } L_i \right) \bmod q.$$

The witness of party i is $\text{lesk}_i = \text{esk}_i \cdot L_i$, for some public known L_i . Notice that $\text{lepk} = \text{epk}$ and $\text{lesk} = \text{esk}$.

This type of linear equality is achieved by the schemes VSS-v2.0 or VSS-v2.1 (and many other variants of Shamir secret sharing), with the Lagrange coefficient $L_i = \prod_{j \in I \setminus \{i\}} (j/(j-i)) \bmod q$. Additionally, we can consider secret sharing schemes based on the Chinese Remainder Theorem [36], [37], where $\text{lesk}_i = \text{esk}_i \cdot M_i$ for $M_i = (M/m_i) \cdot ((M/m_i)^{-1} \bmod m_i) \bmod M$ given the fix product M of all co-primes m_i for $i \in I$ and $q = m_0$. This would be similar to the proof of multi-exponentiation by Yuen et al [38], in the context of single-prover.

MPZKP and ElectionGuard: The proof considered in ElectionGuard is a specific instantiation of our proof, where they use additive homomorphic secret sharing (i.e., VSS-v2.0 and VSS-v2.1, respectively) to describe the secret keys. The substantial differences between our MPZKP and the results of Benaloh et al. [5] are

- **threshold nature** in [5] and *linear* for MPZKP, as they rely on Lagrange coefficients and polynomials to define the relation between guardian keys. Additionally, they report their design to Sparkle threshold signature [39] and claim to be “valuable under threshold conditions”.
- **informal security arguments** for soundness and proof of knowledge, while only formally shown zero-knowledge for MPZKP-with-VSS-v2.1 (by building the simulator). We prove formally all properties for MPZKP and show it also satisfies a new property - *traitor tracing*.

Theorem 5 (Benaloh et al. [5]). *MPZKP-with-VSS-v2.1 satisfies completeness, soundness and zero-knowledge.*

IV. ISSUES AND VULNERABILITIES

In this section we identify and address incorrect statements (made by Benaloh et al. [5]) about the security of the threshold encryption scheme DPKE used in ElectionGuard, and identify a vulnerability in the distributed key generation.

A. Correct results for DPKE and EPKE

We prove the right statements regarding the encryption scheme DPKE used by current variants of ElectionGuard.

EPKE-v2.1.A and EPKE-v2.1.B satisfy IND-1-CCA: EG-v2.0 is based on a trivial variant of DHIES [29], and as such it is IND-CCA2. However, EG-v2.1 considers an additional change where the MAC function is replaced by a Schnorr proof - and the security of this scheme is not claimed

or proven. For completeness, we rely on Theorem 2 to show that they are IND-1-CCA.

DPKE satisfies IND-CPA: The scheme $\text{DPKE} = (\text{KGen}, \text{Enc}, \text{Dec}, \text{Add})$ has not been shown to be IND-CPA or claimed to be in the initial paper [28]. However, it’s assumed to be IND-CPA in [5] when used to design a threshold encryption. For completeness, we show that this construction satisfies IND-CPA.

Theorem 6. *DPKE is IND-CPA under DDH assumption.*

Proof. Similar to ElGamal this reduces to DDH. More precisely, the DDH adversary $\mathcal{B}(g^x, g^y, Z)$ return $(\beta = \beta')$ where the IND-CPA adversary produces $\beta' \leftarrow \mathcal{A}(g^y, (g^x, Z \cdot (g^y)^{m_\beta}))$ for some random choice $\beta \leftarrow \{0, 1\}$. $\square$

DPKE-with-ChPeZKP satisfies IND-1-CCA: We strengthen our previous result by showing how one can obtain an IND-1-CCA variant for the DPKE scheme.

Theorem 7. *DPKE-with-ChPeZKP is IND-1-CCA.*

Proof. In essence, the following result follows naturally from the Theorem 2 and Theorem 6. $\square$

DPKE-with-DKG-v2.0 is threshold IND-CPA: Benaloh et al. [5] (aiming to describe EG-v2.1) don’t prove that DPKE-with-DKG-v2.0 or DPKE-with-DKG-v2.1 are threshold IND-CPA. They show this what they call “ElectionGuard ElGamal encryption” - that in essence is ElGamal. This can be easily observed as in the proof of their Theorem 1 they build the ciphertext as $(c_0, c_1) = (g^r, m_b \cdot \text{pk}^r)$. Moreover, this result has already been established by Cortier et al. [9].

We will show that the proof for DPKE is not straightforward, and requires a bit more than the tactic used in [5]. We provide the proof for DPKE-with-DKG-v2.0, and state that the exact same proof will work for DPKE-with-DKG-v2.1.

Theorem 8. *DPKE-with-DKG-v2.0 is threshold IND-CPA.*

Proof. Let $\mathcal{A}$ be an adversary against threshold IND-CPA and adversary $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ against IND-CPA. We build adversary $(m_0, m_1) \leftarrow \mathcal{B}_1(\mathcal{A}, \text{pk}^*)$ and $\beta' \leftarrow \mathcal{B}_2(\mathcal{A}, c^*)$. Adversary $\mathcal{B}$ wins the IND-CPA experiment if they have $\beta = \beta'$ for $(\text{sk}^*, \text{pk}^*) \leftarrow \text{DPKE.KGen}$, $\beta \leftarrow \{0, 1\}$, and $c^* \leftarrow \text{DPKE.Enc}(\text{pk}^*, m_\beta)$.

Consider that DKG-v2.0 has n participant and threshold t . We split this into honest parties: $\text{Ho} \cup \{n\}$ and corrupted parties Co with $|\text{Co}| < t$. We want to illustrate that the last party n has to be honest and under the control of adversary $\mathcal{B}$. We design adversary $(m_0, m_1) \leftarrow \mathcal{B}_1(\mathcal{A}, \text{pk}^*)$ by

- simulate the distributed key generation for all $i \in \text{Ho} \cup \text{Co}$. If $i \in \text{Co}$ then $\mathcal{A}$ produces publicly pk_i and SchZKP proof π_i , or if $i \in \text{Ho}$ then $\mathcal{B}$ runs a normal key generation and has $(\text{pk}_i, \text{sk}_i)$ and a valid SchZKP proof π_i .
- on party’s n turn the adversary $\mathcal{B}_1$ recover the secret keys $\{\text{sk}_i | i \in \text{Co}\}$ by rewinding $\mathcal{A}$ such it provides a different challenge response (e.g., different output to the hash function) for SchZKP proofs. Then, it sets the key $\text{pk}_n = \text{pk}^*$ and simulates shares $\text{sh}_{n,i}$ for $i \in \text{Co}$ such

that $g^{\text{sh}_{n,i}} = (\text{pk}^*)^i \cdot \prod_{j=1}^{t-1} g^{a_j \cdot i^j}$ for some choices of $a_1, \dots, a_{t-1}$.

- adversary $\mathcal{A}$ will be challenged on the public key $\text{epk} = \text{pk}^* \cdot \prod_{i \in \text{HoUCo}} \text{pk}_i$ and the matching secret key $\text{esk} = \text{sk}^* \cdot \sum_{i \in \text{HoUCo}} \text{sk}_i$ - where the adversary $\mathcal{B}_1$ doesn't know sk^* . For simplicity, we consider:

$$\begin{aligned} \text{ask} &= \sum_{i \in \text{HoUCo}} \text{sk}_i & \text{epk} &= \text{apk} \cdot \text{pk}^* \\ \text{apk} &= \prod_{i \in \text{HoUCo}} \text{pk}_i & \text{esk} &= \text{ask} + \text{sk}^* \end{aligned}$$

- adversary $\mathcal{B}$ returns $(m_0, m_1) \leftarrow \mathcal{A}(\text{epk})$.

We design adversary $\beta' \leftarrow \mathcal{B}_2(\mathcal{A}, c^*)$ by:

- the aim is to run $\gamma' \leftarrow \mathcal{A}(\text{cip}^*)$ by producing the challenge ciphertext $(\text{cip}_0^*, \text{cip}_1^*) \leftarrow \text{cip}^*$ from $(c_0^*, c_1^*) \leftarrow c^*$. We randomly select $\gamma \leftarrow \{0, 1\}$ and compute

$$(\text{cip}_0^*, \text{cip}_1^*) \leftarrow (c_0^*, c_1^* \cdot c_0^{\text{ask}} \cdot g^{\text{ask} \cdot m_\gamma}).$$

This is equivalent to computing the following relations

$$\begin{aligned} c_0^* &= g^r \text{ and } c_1^* = g^{\text{sk}^* \cdot (r+m_\beta)} \\ \text{cip}_0^* &= g^r \text{ and } \text{cip}_1^* = g^{\text{sk}^* \cdot (r+m_\beta)} \cdot (g^{\text{ask}})^{r+m_\gamma} \end{aligned}$$

- return the outcome of $\beta' \leftarrow (\gamma' = \gamma)$. The intuition here is that adversary $\mathcal{A}$ could have only guessed correctly if cip^* is a valid challenge ciphertext ($\beta = \gamma$), otherwise it would have just made a random guess. That means, that adversary $\mathcal{B}$ only has half of the advantage of $\mathcal{A}$.

Based on the fact that we can always build an adversary $\mathcal{B}$ against IND-CPA with a similar advantage as $\mathcal{A}$ and that any adversary $\mathcal{B}$ will always have a negligible advantage implies the result of this theorem. $\square$

More details on building the reduction to the IND-CPA security above are found in Appendix F.

DPKE with DKG-v2.0 and ChPeZKP is threshold IND-1-CCA: We strengthen our results on DPKE by showing that if one combines this scheme with a verifiable secret sharing and a proof then they obtain threshold IND-1-CCA. Our result will also hold for DKG-v2.1.

Theorem 9. *DPKE-with-DKG-v2.0-and-ChPeZKP is threshold IND-1-CCA.*

Proof. Consider $\text{DPKE}^+ = \text{DPKE-with-DKG-v2.0}$ and $\text{DPKE}^{++} = \text{DPKE-with-DKG-v2.0-and-ChPeZKP}$.

The encryption scheme DPKE^{++} produces ciphertexts (cip, π) with $\text{cip} \leftarrow \text{DPKE}^+.\text{Enc}$ and $\pi \leftarrow \text{ChPeZKP}$. We follow the procedure in Theorem 3 and leverage the extractor in ChPeZKP such that we can produce the witness for the ciphertext cip (where the witness is the randomness and not the secret key). This is straightforward as the pair (cip, π) is produced by a single party (i.e., the voter). The extractor works even on proofs produced by the adversary - it relies on rewinding the adversary and then producing a different value during the challenge. The result follows based on Theorem 7. $\square$

B. Vulnerability: lack of authentication in DKG

As stated by Benaloh et al. [5], [6] the distributed key generation is done without signing keys, no PKI infrastructure, and under the recommendation of in-person voting. Essentially, the DKG is run in-person and deemed successful if all guardians obtain the same hash output - H_G in EG-v2.1 as it contains the agreed public key epk and the list of all communication transcripts, or H_E in EG-v2.0 with only the hash over epk .

Naturally, the DKG will be run as many times as it needs to ensure agreement, and it might never terminate. When DKG fails they consider an out-of-bound dispute resolution mechanism, and attempt to identify and ban misbehaving guardians. No details on the dispute-resolution are available.

Another interest point stated by Benaloh et al. [5] is that the DKG is run between “publicly identifiable parties, and there is little benefit for a malicious participant”.

Misbehaving guardians for in-person DKG: We want to highlight that it is possible to have guardians misbehave and not be detected. Due to the lack of authentication in the share exchange phase, any internal party can severely disrupt this process by sending false shares (under another guardian's public key). We assume that the in-person DKG is run without direct internet connection (e.g. in an isolated LAN or by physically exchanging USB sticks), to trivially prevent external attackers from sending false information.

We anticipate the aim of malicious party is to delay/disrupt the election process or try to frame honest parties. We argue that running the DKG with a malicious party will decrease trust in the election and lead to a process that will never terminate. Due to the lack of authentication, it's not possible to resolve *non-intrusively* the following situation: guardian G_i receives a shares from G_j and another share from someone pretending to be G_j . Naturally, one can address this *intrusively* - by scanning everything in the devices the guardians and monitor traffic between them, however, we don't advocate such a drastic approach in what is already a high stress and intense electoral environment.

While EG-v2.1 aims to moderate this aspect by having guardians check the hash of the entire transcript, EG-v2.0 (still a valid system) only hashes the public key. The code⁵ seems to be a few years behind the specifications, and it's unclear what version is publicly available.

To address this, the system needs (1) to correlate public keys with identify in a preliminary phase (a PKI infrastructure), and (2) have an authenticated channel (e.g., by *sign-then-encrypt* [31]). In an in-person setup, one can take advantage of the “publicly identifiable” aspect of guardians to ensure (1) before it runs DKG.

ElectionGuard and online DKG: ElectionGuard seems to allow an *online* DKG for internet voting, that is run “in a manner similar to Helios” Benaloh et al. [5].

Helios considers an *online* DKG [40] (see Section 11.2.2.1) where guardians are given an URL link to upload their public keys, thus setting the PKI infrastructure. There are no shares

⁵<https://github.com/Election-Tech-Initiative/electionguard>

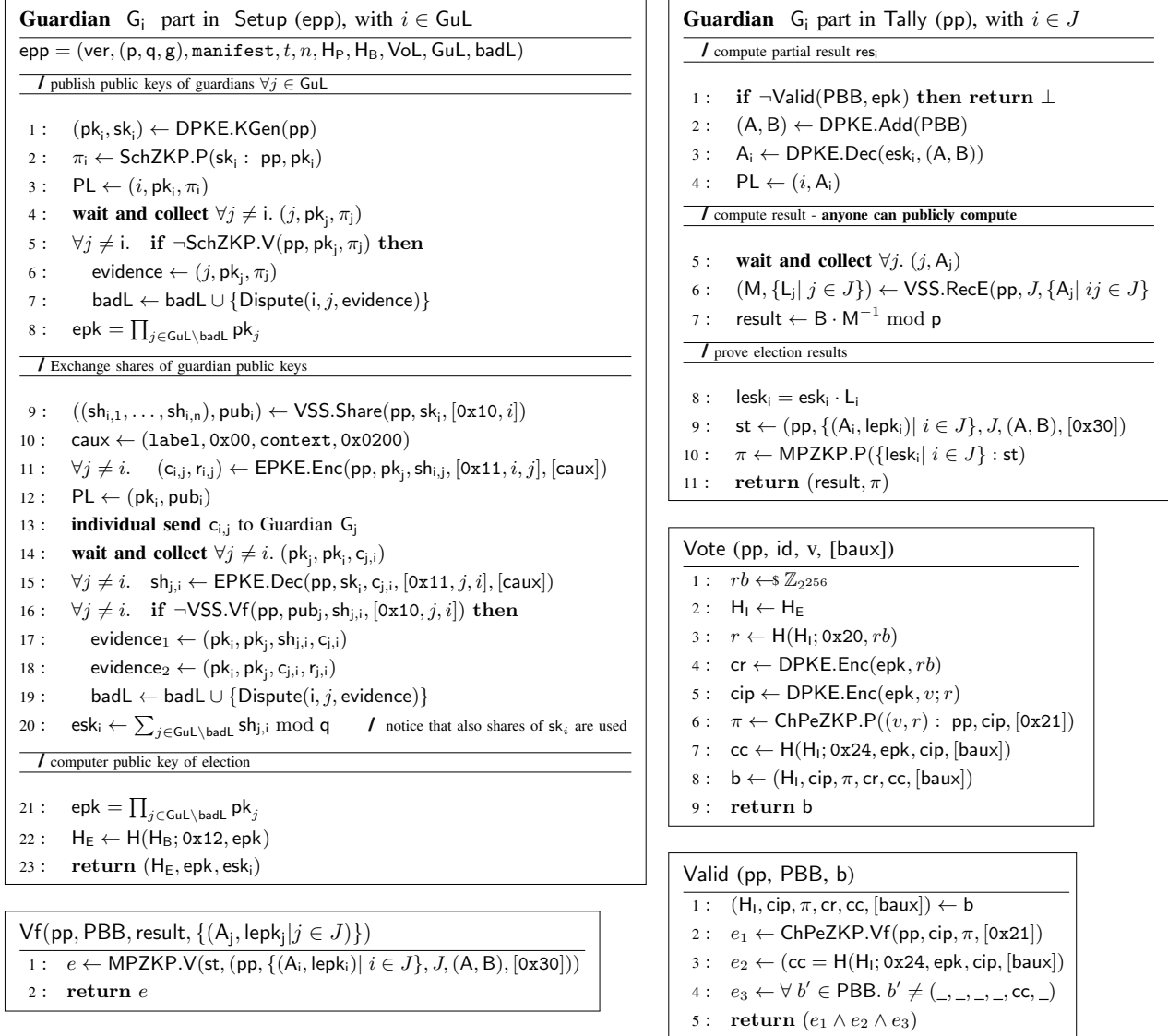


Fig. 4. ElectionGuard EG-v2.0 general description.

exchanged here, as all guardian public keys are combined in one single election key - all guardians are needed for decryption. Authentication is guaranteed by the individual links sent to guardians.

We want to highlight the need to update the ElectionGuard DKG where authenticated channels are used, to prevent internal or external attackers from impersonating guardians. We believe most organizations or software designers will identify the need for a PKI infrastructure when doing an online DKG, but will be unaware of the security flaw based on the lack of authentication.

We've seen a drive in recent years to implement digital identity and digital wallets⁶, simulating the success of Estonia, including in e-voting⁷. This naturally integrates all citizens

in a PKI infrastructure and may empower governments to consider or run electronic elections.

V. FORMALIZATION OF ELECTIONGUARD

In this section, we produce an algorithmic formalization of ElectionGuard = (Setup, Vote, Valid, Tally) for the current variants EG-v2.0 and EG-v2.1. We describe the interactive Setup and Tally from the view of individual parties.

This is the first rigorous formalization as the documentation focuses mainly on implementation details and Benaloh et al. [5] are informal and imprecise. Our formalization will be useful to all researcher that wish to use ElectionGuard and/or reason formally on its security and privacy; such a formalization is a stepping stone for all future endeavors.

We consider the following entities and their role

- **Election Authority** EA sets the election manifest that contains election information, e.g., list of candidates, list

⁶https://commission.europa.eu/topics/digital-economy-and-society/european-digital-identity_en

⁷<https://www.id.ee/en/article/e-voting-and-e-elections/>

of eligible voters VoL, list of guardians GuL, and quorum t . We consider a threshold model (t, n) with $0 < t \leq n$ and $|\text{GuL}| = n$. We maintain a list of banned guardians badL that is populated during the setup process.

- **Guardians** $\{G_i(\text{epk}_i, \text{esk}_i) \mid i \in \text{GuL}\}$ own a share of the election secret key esk, such a quorum can produce the election result and ensure correctness of the process while maintaining privacy. They provide the election public key $\text{epk} = \prod_{j \in \text{GuL} \setminus \text{badL}} \text{epk}_j$.
- **Voters** $\{\text{id}_j \mid j \in \text{VoL}\}$ cast their vote $v \in \{0, 1\}$ - in a simple referendum style election, but our results can be generalized for multiple types of elections.
- **Election Public Information** PL stores publicly verifiable information like the threshold model (t, n) , guardian public keys, election public keys, and the manifest. This is a simple and elegant way of broadcasting.
- **Election Board** PBB stores only ballots cast during the election. This is disjoint for ease of presentation from PL but can be run by the same entities.

Building blocks: Before the election starts, the following information is agreed by EA: $(\text{ver}, (p, q, g), \text{manifest}, t, n)$, where $\text{ver} \in \{v2.0, v2.1\}$, $(p, q, g) \leftarrow \$ \text{GSetup}$, and (t, n) - the threshold and total number of guardians. The election authority EA produces the election hashes $(H_P, H_B) \leftarrow \text{GenHash}(\text{epp})$ and defines the election public parameters pp as $(\text{ver}, (p, q, g), \text{manifest}, t, n, H_P, H_B, \text{VoL}, \text{GuL}, \text{badL})$.

GenHash(epp) with $\text{epp} = (\text{ver}, (p, q, g), \text{manifest}, t, n)$		
1:	$H_P \leftarrow H(\text{ver}; 0x00, (p, q, g), n, t)$	/ Parameter Base Hash
2:	$H_B \leftarrow H(H_P; 0x01, \text{manifest})$	/ Election Base Hash
3:	return (H_P, H_B)	

The system uses encryption schemes DPKE and EPKE, the zero-knowledge proofs SchZKP, ChPeZKP, MPZKP, and the secret sharing scheme VSS. The proof SchZKP and encryption EPKE use an $\text{hkey} = H_P$ in the Setup algorithm - this is the key for the hash function. We have $\text{ChPeZKP.hkey} = H_I$ and $\text{MPZKP.hkey} = H_E$, for H_E defined at the end of Setup and H_I produced in Vote.

A. Distributed Key Generation

The aim is to generate election keys $(\text{epk}, \{\text{esk}_j \mid j \in \text{GuL} \setminus \text{badL}\})$ such that there is one common public election key epk and each guardian G_j receives a share esk_j of the corresponding secret key esk, for $j \in \text{GuL} \setminus \text{badL}$. In essence, the process should be equivalent to running

$$(\text{epk}, \{\text{esk}_j \mid j \in \text{GuL} \setminus \text{badL}\}, \text{pub}) \leftarrow \text{VSS.Share}(\text{pp}, \text{esk})$$

for some value $\text{esk} \in \mathbb{Z}_q$. This is achieved by each guardian running its own secret sharing procedure and then combining all shares received like in the additive homomorphic secret sharing process (see Section II)

$$\text{Share}(\text{pp}, (\sum_{j \in \text{GuL} \setminus \text{badL}} \text{sk}_i)) = \sum_{j \in \text{GuL} \setminus \text{badL}} \text{Share}(\text{pp}, \text{sk}_i).$$

To strengthen this procedure the system runs verifiable secret sharing VSS such that each party can verify that they've

received the correct share. Guardian G_i sends share $\text{sh}_{i,j}$ to G_j by encrypting it with EPKE under the public key of G_j . If complains appear in this stage, a dispute-resolution mechanism is employed to identify the guilty party. In EG-v2.0 and EG-v2.1 versions, it is straightforward to see if enough accusations are introduced in the system then this substantially weakness the security of the key (as shares of keys are divulged during the dispute resolution process), thus the setup must be run with fresh values for all $\text{GuL} \setminus \text{badL}$.

Remark 2 (Knowledge of pk_i). EG-v2.0 and EG-v2.1 rely on the assumption that the public keys $\{\text{pk}_i \mid i \in \text{GuL}\}$ are linked in a known way to the guardians - so that anyone communicating with G_i knows their public key pk_i . In this case, the lines 1-7 from Setup can be dropped. However, if we consider the need for an authenticated channel - and consider the authenticated channel as an established link of identity, then we need lines 1-7. This would be significant as for each complain/dispute the system needs to regenerate keys and redo lines 1-7, thus re-issuing $(\text{sk}_i, \text{pk}_i)$.

B. Voting and Validating

The voting process relies on the paradigm Encrypt-and-PoK (see Section II-D) where the vote $v \in \{0, 1\}$ is encrypted using DPKE and made non-malleable using ChPeZKP. Additionally, the system preserves the randomness used such that it can be audited at a later stage - naturally, the randomness is stored encrypted under epk⁸.

The ballot has the form $b \leftarrow (H_I, \text{cip}, \pi, \text{cr}, \text{cc}, [\text{baux}])$ with H_I being a ballot identifier, (cip, π) are the encryption and proof for the vote, cr is the encryption of the randomness that could be useful during risk-limit audits, and the confirmation code $\text{cc} \leftarrow H(H_I; 0x24, \text{epk}, \text{cip}, [\text{baux}])$. The auxiliary information [baux] provides context if this ballot should be linked with previous ballots (either cast by same voter, or produced by same machine). We consider no revoting with $\text{baux} \leftarrow \perp$ and have kept it in the formalization for future models.

The system only accepts valid ballots - and this is enforced by checking all proofs in the ballot and not allowing duplicate ciphertexts (i.e., by having unique confirmation codes). It has been established by Cortier et al. [41] that allowing for duplicate ballots leads to trivially breaking privacy - as the adversary just needs to cast many identical ciphertexts and infer from the election result.

C. Tally

The system relies on a threshold model where a quorum of guardians $t \leq |J|$ produce the election result result. The process relies on using partial decryptions DPKE.Dec and secret sharing reconstruction in the exponent VSS.RecE over a single ciphertext (A, B) - obtained from homomorphically adding all valid ciphertexts in PBB.

To ensure correctness of the result the multi-prover zero-knowledge system MPZKP is used. Additionally, it provides

⁸In [4] the Setup is run twice to produce two election keys epk and $\widehat{\text{epk}}$, with epk used to encrypt votes and $\widehat{\text{epk}}$ to encrypt the randomness.

traitor tracing capabilities to identify what guardian has deviated from the protocol.

D. Differences between EG-v2.0 and EG-v2.1

The description in Figure 4 fits perfectly with EG-v2.0 where the secret sharing scheme $VSS = VSS\text{-}v2.0$ and the encryption scheme is $EPKE = EPKE\text{-}v2.0$.

For EG-v2.1 we consider $VSS = VSS\text{-}v2.1$, $EPKE \in \{EPKE\text{-}v2.1.A \text{ (in Setup)}, EPKE\text{-}v2.1.B \text{ (in Vote)}\}$ and some slight changes to the Setup and H_I format. The main difference comes from using a different public key $\widehat{epk}$ to encrypt the randomness in Vote: $cr \leftarrow EPKE\text{-}v2.1.A.\text{Enc}(\widehat{epk}, rb)$. To generate key $\widehat{epk}$ and matching guardian secret keys $\{\widehat{esk}_j | j \in GuL \setminus badL\}$ the exchange shares procedure in Setup (line 10) is run twice and both shares are sent using EPKE-v2.1.B. For completeness, we present EG-v2.1.Setup in Appendix D⁹.

Notice that H_G - the hash of entire public transcript, implies that epk and $\widehat{epk}$ use the same guardian list $GuL \setminus badL$. Additionally, each ballot has a unique identifier $H_I \leftarrow H(H_E; 0x20, idb)$ (for $idb \leftarrow \mathbb{Z}_{2^{256}}$) that is used later in the risk-limit auditing process. This choice doesn't have an impact on the verifiability and privacy as only the quorum of guardians can extract the information.

E. Substantial Endeavor to Formalize

We want to highlight that it was a substantial task to formalize ElectionGuard, mostly due to the minimal referencing of core cryptographic primitives and the use of bespoke (unproven) primitives. We list the many items that we had to identify, formalize, investigate or analyze:

- DPKE and it's security guarantees: IND-CPA, threshold IND-CPA, and threshold IND-1-CCA. See next section for an in-depth discussion on this.
- EPKE-v2.0, EPKE-v2.1.A, and EPKE-v2.1.B, together with their security guarantees. EPKE-v2.0 is as a variant of DHIES [29] and inherits its IND-CCA2 security. Additionally, we show that EPKE-v2.1.A and EPKE-v2.1.B are IND-1-CCA.
- VSS-v2.0 and VSS-v2.1 and their many security guarantees, that go beyond Feldman [21].
- Vulnerability in DKG - no authenticated channel.
- a novel multi-prover zero-knowledge. Benaloh et al [5] propose an in-depth description and security proof for a threshold multi-prover system. We have shown that a simpler concept will suffice thus opening possibilities for more efficient alternatives, even post-quantum ones.

VI. THRESHOLD PRIVACY

We propose a definition of privacy in the threshold context, and apply it to ElectionGuard. To the best of our knowledge, this is the first time an *accurate* formal definition of threshold privacy for voting is explicitly stated. Often formalizations and

(formal or machine-checked) proofs are based on a trusted single party with an informal argument over the threshold aspect of DKG. We rely on the result of Cortier et al. [10] that have shown that voter privacy can be obtained by looking at three properties: *ballot privacy*, *strong correctness*, and *strong consistency*.

We acknowledge that Cortier et al. [9] introduced the first formal definition of threshold ballot privacy, and applied it to threshold Helios (that is identical to ElectionGuard v.0.85). However, their definition is weaker due to:

- *handling of corrupt guardian keys*. Their model has the experiment generate all guardian keys and then allow the adversary to corrupt guardians and obtain secret keys. This has the benefit of the experiment easily producing the tally by itself - as it has all secret keys. This is incorrect, as in a threshold model the decryption is done by individual parties - essentially, the adversary produces something in the tally on behalf of the guardians it controls. This is modeled in our definition using an interactive Tally, and additionally the experiment does not decide on the secret keys for the guardians the adversary corrupts - so Setup is also interactive.
- *casting ballots*. Original definition is unclear how the adversary $\mathcal{A}(PBB_\beta)$ is expected to produce $PBB_{1-\beta}$ without oracle access. This is addressed later by Cortier et al. [10] for privacy with a single-guardian.

Unchanged strong correctness and strong consistency:

Strong correctness looks at ensuring any ballot created using Vote will always satisfy the checks in Valid. ElectionGuard trivially satisfies this as relies on completeness of ChPeZKP and the collision resistance of the hash function H . Notice that while the adversary can control the ballot box PBB, they would need to predict the output of the hash function cc to prevent future ciphertexts from being introduced. Strong consistency looks at ensuring the election result from Tally is correct - basically, it's identical to simply counting all the votes cast. ElectionGuard naturally satisfies this property due to the correctness of the decryption DPKE and soundness of the MPZKP proof. Some e-voting schemes allow for revoting but not in our formalization of ElectionGuard as it's used in real-world statutory elections.

New threshold ballot privacy: To account for the threshold design of the Setup and Tally we consider *threshold ballot privacy* (TBPRIV) where the adversary controls some of the guardians - we formalize our definition in Figure 5. In TBPRIV the adversary $\mathcal{A}$ aims to distinguish which of two valid votes $(v_0, v_1) \in \{0, 1\}^2$ is cast by seeing a ballot for one of them $b \leftarrow \text{Vote}(pp, v_\beta)$. This is modeled considering left-right style of experiments, where a fixed value $\beta \in \{0, 1\}$ decides what variant of the experiment is running and the adversary needs to produce a guess β' . As shown in Figure 5, the adversary has a number of oracle access to simulate normal actions in an election. What makes this definition different from Cortier et al. [10] is that the oracles $O\text{Setup}^*$ and $O\text{Tally}^*$ are run in a distributed way with adversary controlling the parties $j \in Co$. Notice that the experiment does not have access

⁹The EG-v2.1.Setup additionally generates a separate communication key $(\text{compk}_i, \text{comsk}_i) \leftarrow \text{DPKE.KGen}(pp)$ used to encrypt the shares. However, this doesn't seem to have an impact on the security guarantees as compk_i is assumed to be linked to pk_i, pk_i .

$\text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}}^{\text{tbpriv}, \beta}(\text{epp})$ 1: $\text{PBB}_0, \text{PBB}_1, \text{Voted} \leftarrow \emptyset$ 2: $\text{callS}, \text{callT} \leftarrow \text{false}$ 3: $\text{epp} \leftarrow \text{GenHash}(\text{epp})$ 4: $\beta' \leftarrow \mathcal{A}^O(\text{epp})$ 5: return $\beta' \wedge \text{Co} < t$	$\text{OVote}(\text{id}, v_0, v_1)$ with $(v_0, v_1) \in \{0, 1\}^2$ 1: if $(\text{id} \in \text{Ho} \wedge \text{id} \notin \text{Voted})$ then 2: $b_0 \leftarrow \text{Vote}(\text{pp}, \text{id}, v_0)$ 3: $b_1 \leftarrow \text{Vote}(\text{pp}, \text{id}, v_1)$ 4: $\text{Voted} \leftarrow \text{Voted} \cup \{\text{id}\}$ 5: if $(\text{Valid}(\text{pp}, \text{PBB}_\beta, b_\beta))$ then 6: $\text{PBB}_0 \leftarrow \text{PBB}_0 \cup \{b_0\}$ 7: $\text{PBB}_1 \leftarrow \text{PBB}_1 \cup \{b_1\}$	$\text{OTally}^*(J)$ for $\beta = 0$ 1: if callT then return $\perp$ 2: $(r, \Pi) \leftarrow \text{Tally}(\text{PBB}_0, \{\text{esk}_j \mid j \in J \cap \text{Ho}\})$ 3: $\text{callT} \leftarrow \text{true}$ 4: return (r, Π) $\text{OTally}^*(J)$ for $\beta = 1$ 1: if callT then return $\perp$ 2: $(\text{res}, _) \leftarrow \text{Tally}(\text{PBB}_0, \{\text{esk}_j \mid j \in J \cap \text{Ho}\})$ 3: $\Pi' \leftarrow \text{Sim}(\text{pp}, \text{PBB}_1, \text{res})$ 4: $\text{callT} \leftarrow \text{true}$ 5: return (r, Π') $\text{OPub}()$ 1: return PBB_β
$\text{OSetup}^*(\text{epp})$ 1: if callS then return $\perp$ 2: $(\text{pp}, \{\text{esk}_j \mid j \in \text{Ho}\}) \leftarrow \text{Setup}(\text{epp})$ 3: $\text{callS} \leftarrow \text{true}$ 4: return pp	$\text{OCast}(\text{id}, b)$ 1: $e \leftarrow \text{Valid}(\text{pp}, \text{PBB}_\beta, b)$ 2: if $(\text{id} \in \text{Co} \wedge e \wedge \text{id} \notin \text{Voted})$ then 3: $\text{Voted} \leftarrow \text{Voted} \cup \{\text{id}\}$ 4: $\text{PBB}_0 \leftarrow \text{PBB}_0 \cup \{b\}$ 5: $\text{PBB}_1 \leftarrow \text{PBB}_1 \cup \{b\}$	

Fig. 5. Threshold Ballot Privacy Experiment and Oracles

to the secret keys of parties $j \in \text{Co}$ - this is a more realistic formalization that current definitions of *threshold receipt freeness* [42] and *threshold privacy* [9], where the system has access to all secret keys. The advantage of the adversary $\mathcal{A}$ is negligible:

$$\text{Adv}_{\mathcal{A}, \mathcal{V}, \text{Sim}}^{\text{tbpriv}}(\lambda) = \left| \Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}}^{\text{tbpriv}, 0} = 1] - \Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}}^{\text{tbpriv}, 1} = 1] \right|.$$

Constraints for ElectionGuard: We consider that in ElectionGuard voters vote only once (i.e., $\text{id} \notin \text{Voted}$ in TBPRIV). We consider two common mods for the system: online and in-person. The in-person mode is the recommended option by ElectionGuard as it provides a safe and secure space for voters to cast their vote, while for the online in addition to authentication issues there could be many other ways the adversary can disrupt or coerce the voter. When looking only at privacy, the fact that the adversary can impersonate voters (via controlling their authentication during casting) does not impact the guarantees of the system - and as such many definitions (including by Cortier et al. [10]) do not consider this authentication. Verification and receipt-freeness require restricting the adversary such that it can't control the authentication of honest voters.

EG-v2.0 and EG-v2.0 are threshold private: We prove this for the DKG where authentication has been fixed. Otherwise, the adversary can trivially remove honest parties by sending them fake shares and honest parties complain and abort. We prove TBPRIV for EG-v2.0, as it already trivially satisfies strong correctness and strong consistency. The proof for EG-v2.1 is similar.

Theorem 10. EG-v2.0 is TBPRIV.

Proof. Consider $\text{Game}_0 = \text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}}^{\text{tbpriv}, 0}$, and $\text{Game}_3 = \text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}}^{\text{tbpriv}, 1}$. We prove this without ROM access, and rely on the result of Nguyen et al. [15] - otherwise, we would have also updated the OSetup^* .

We introduce Game_1 as Game_0 where we check if the relation for MPZKP holds. This is done by introducing in the oracle OTally^* an extractor $\{(i, \text{esk}_i) \mid i \in$

$J\} \leftarrow \text{Ext}$ and evaluating MPZKP.R. If the relation is false, then we do not produce any proof. Due to the proof of knowledge property of MPZKP, we have:

$$|\Pr[\text{Game}_0 = 1] - \Pr[\text{Game}_1 = 1]| \leq \text{negl}(\lambda).$$

We introduce Game_2 as Game_1 where the MPZKP proof in the OTally^* is replaced with a simulated proof $\Pi' \leftarrow \text{Sim}(\text{pp}, \text{PBB}_0, \text{res})$. It is important that this is done only when the relation holds, and as such prior to this we had to evaluate the relation. Due to the zero-knowledge property of MPZKP, we have:

$$|\Pr[\text{Game}_1 = 1] - \Pr[\text{Game}_2 = 1]| \leq \text{negl}(\lambda).$$

We now rely on Theorem 9 to show that the games Game_3 as Game_2 are indistinguishable due to the threshold IND-1-CCA property of the encryption scheme:

$$|\Pr[\text{Game}_2 = 1] - \Pr[\text{Game}_3 = 1]| \leq \text{negl}(\lambda). \quad \square$$

VII. CONCLUSIONS

In this paper we have achieved an accurate algorithmic formalization of the current ElectionGuard variants - EG-v2.0 and EG-v2.1, together with a proof of their threshold privacy. As part of that we investigated and addressed vulnerabilities and corrected previous security claims. Additionally, we proposed a novel type of multi-prover non-interactive zero-knowledge proofs and shown how they straightforwardly instantiate with the proof used in ElectionGuard. As future work, it would be interesting to provide machine-checked proofs for privacy and verifiability using our formalization of ElectionGuard, or to design a post-quantum version of ElectionGuard by relying on our simpler multi-prover zero-knowledge proof.

REFERENCES

- [1] M. Bernhard, J. Benaloh, J. A. Halderman, R. L. Rivest, P. Y. A. Ryan, P. B. Stark, V. Teague, P. L. Vora, and D. S. Wallach, "Public Evidence from Secret Ballots," in *Proceedings of International Joint Conference on Electronic Voting - E-Vote-ID 2017*, ser. Lecture Notes in Computer Science, R. Krimmer, M. Volkamer, N. B. Binder, N. Kersting, O. Pereira, and C. Schürmann, Eds., vol. 10615. Springer, 2017, pp. 84–109. [Online]. Available: https://doi.org/10.1007/978-3-319-68687-5_6
- [2] P. Y. A. Ryan, P. B. Rønne, and V. Iovino, "Selene: Voting with transparent verifiability and coercion-mitigation," in *Proceedings of Financial Cryptography and Data Security - FC 2016*, ser. Lecture Notes in Computer Science, J. Clark, S. Meiklejohn, P. Y. A. Ryan, D. S. Wallach, M. Brenner, and K. Rohloff, Eds., vol. 9604. Springer, 2016, pp. 176–192. [Online]. Available: https://doi.org/10.1007/978-3-662-53357-4_12
- [3] J. Benaloh and M. Naehrig, "ElectionGuard design specification version 2.0.0." <https://www.electionguard.vote/spec/>.
- [4] J. Benaloh, M. Naehrig, and O. Pereira, "ElectionGuard design specification version 2.1.0." <https://www.electionguard.vote/spec/>.
- [5] J. Benaloh, M. Naehrig, O. Pereira, and D. S. Wallach, "ElectionGuard: a Cryptographic Toolkit to Enable Verifiable Elections," in *Proceedings of USENIX Security 2024*, D. Balzarotti and W. Xu, Eds. USENIX Association, 2024. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/benaloh>
- [6] J. Benaloh, M. Naehrig, and O. Pereira, "REACTIVE: Rethinking effective approaches concerning trustees in verifiable elections," Cryptology ePrint Archive, Paper 2024/915, 2024. [Online]. Available: <https://eprint.iacr.org/2024/915>
- [7] Y. Chen and Y. Lindell, "Feldman's Verifiable Secret Sharing for a Dishonest Majority," *IACR Commun. Cryptol.*, vol. 1, no. 1, p. 16, 2024. [Online]. Available: <https://doi.org/10.62056/ak2isgvtw>
- [8] T. Haines, R. Goré, and J. Stodart, "Machine-checking the universal verifiability of electionguard," in *Proceedings of Secure IT Systems - NordSec 2020*, ser. Lecture Notes in Computer Science, Mikael Asplund and Simin Nadjm-Tehrani, Ed., vol. 12556. Springer, 2020, pp. 57–73. [Online]. Available: https://doi.org/10.1007/978-3-030-70852-8_4
- [9] V. Cortier, D. Galindo, S. Glondu, and M. Izabachène, "Distributed elgamal à la pedersen: Application to helios," in *Proceedings of ACM Workshop on Privacy in the Electronic Society - WPES 2013*, A. Sadeghi and S. Foresti, Eds. ACM, 2013, pp. 131–142. [Online]. Available: <https://doi.org/10.1145/2517840.2517852>
- [10] D. Bernhard, V. Cortier, D. Galindo, O. Pereira, and B. Warinschi, "SoK: A Comprehensive Analysis of Game-Based Ballot Privacy Definitions," in *Proceedings of IEEE Symposium on Security and Privacy - IEEE SP 2015*. IEEE Computer Society, 2015, pp. 499–516. [Online]. Available: <https://doi.org/10.1109/SP.2015.37>
- [11] Y. Dodis, T. Ristenpart, J. P. Steinberger, and S. Tessaro, "To Hash or Not to Hash Again? (In)Differentiability Results for H 2 and HMAC," in *Proceedings of Advances in Cryptology - CRYPTO 2012*, ser. Lecture Notes in Computer Science, R. Safavi-Naini and R. Canetti, Eds., vol. 7417. Springer, 2012, pp. 348–366. [Online]. Available: https://doi.org/10.1007/978-3-642-32009-5_21
- [12] A. Chiesa and E. Yogev, *Building Cryptographic Proofs from Hash Functions*, 2024, online, Accessed 20 January 2026. [Online]. Available: <https://github.com/hash-based-snargs-book>
- [13] N. Athamnah, E. Florentz-Konopnicki, and R. D. Rothblum, "Rate-1 Zero-Knowledge Proofs from One-Way Functions," in *Proceedings of Theory of Cryptography - TCC 2024, Part I*, ser. Lecture Notes in Computer Science, E. Boyle and M. Mahmoody, Eds., vol. 15364. Springer, 2024, pp. 319–350. [Online]. Available: https://doi.org/10.1007/978-3-031-78011-0_11
- [14] M. Chase and A. Lysyanskaya, "On Signatures of Knowledge," Cryptology ePrint Archive, Paper 2006/184, 2006. [Online]. Available: <https://eprint.iacr.org/2006/184>
- [15] N. K. Nguyen, E. Theodorakis, and B. Warinschi, "Lifting Standard Model Reductions to Common Setup Assumptions," in *Proceedings of Public-Key Cryptography - PKC 2022, Part II*, ser. Lecture Notes in Computer Science, G. Hanaoka, J. Shikata, and Y. Watanabe, Eds., vol. 13178. Springer, 2022, pp. 130–160. [Online]. Available: https://doi.org/10.1007/978-3-030-97131-1_5
- [16] D. Bernhard, O. Pereira, and B. Warinschi, "How Not to Prove Yourself: Pitfalls of the Fiat-Shamir Heuristic and Applications to Helios," in *Proceedings of Advances in Cryptology - ASIACRYPT 2012*, ser. Lecture Notes in Computer Science, X. Wang and K. Sako, Eds., vol. 7658. Springer, 2012, pp. 626–643. [Online]. Available: https://doi.org/10.1007/978-3-642-34961-4_38
- [17] F. Hao, "Schnorr Non-interactive Zero-Knowledge Proof," Internet Engineering Task Force Documents - RFC 8235, 2017.
- [18] M. Backes, A. Kate, and A. Patra, "Computational Verifiable Secret Sharing Revisited," in *Proceedings of Advances in Cryptology - ASIACRYPT 2011*, ser. LNCS, D. H. Lee and X. Wang, Eds., vol. 7073. Springer, 2011, pp. 590–609. [Online]. Available: https://doi.org/10.1007/978-3-642-25385-0_32
- [19] T. P. Pedersen, "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing," in *Proceedings of Advances in Cryptology - CRYPTO 1991*, ser. Lecture Notes in Computer Science, J. Feigenbaum, Ed., vol. 576. Springer, 1991, pp. 129–140. [Online]. Available: https://doi.org/10.1007/3-540-46766-1_9
- [20] A. Shamir, "How to Share a Secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979. [Online]. Available: <https://doi.org/10.1145/359168.359176>
- [21] P. Feldman, "A Practical Scheme for Non-interactive Verifiable Secret Sharing," in *Proceedings of Annual Symposium on Foundations of Computer Science - FOCS 1987*. IEEE Computer Society, 1987, pp. 427–437. [Online]. Available: <https://doi.org/10.1109/SFCS.1987.4>
- [22] J. C. Benaloh, "Secret Sharing Homomorphisms: Keeping Shares of A Secret Sharing," in *Proceedings of Advances in Cryptology - CRYPTO 1986*, ser. Lecture Notes in Computer Science, A. M. Odlyzko, Ed., vol. 263. Springer, 1986, pp. 251–260. [Online]. Available: https://doi.org/10.1007/3-540-47721-7_19
- [23] D. Bernhard, "Zero-Knowledge Proofs in Theory and Practice," Ph.D. dissertation, University of Bristol, 2014.
- [24] V. Cortier, C. C. Dragan, F. Dupressoir, B. Schmidt, P. Strub, and B. Warinschi, "Machine-Checked Proofs of Privacy for Electronic Voting Protocols," in *Proceedings of IEEE Symposium on Security and Privacy, SP 2017*. IEEE Computer Society, 2017, pp. 993–1008. [Online]. Available: <https://doi.org/10.1109/SP.2017.28>
- [25] M. Bellare, A. Desai, D. Pointcheval, and P. Rogaway, "Relations Among Notions of Security for Public-Key Encryption Schemes," in *Proceedings of Advances in Cryptology - CRYPTO 1998*, ser. Lecture Notes in Computer Science, H. Krawczyk, Ed., vol. 1462. Springer, 1998, pp. 26–45. [Online]. Available: <https://doi.org/10.1007/BFb0055718>
- [26] Y. Tsiounis and M. Yung, "On the Security of ElGamal Based Encryption," in *Proceedings of Public Key Cryptography - PKC 1998*, ser. Lecture Notes in Computer Science, H. Imai and Y. Zheng, Eds., vol. 1431. Springer, 1998, pp. 117–134. [Online]. Available: <https://doi.org/10.1007/BFb0054019>
- [27] C. Schnorr and M. Jakobsson, "Security of Signed ElGamal Encryption," in *Proceedings of Advances in Cryptology - ASIACRYPT 2000*, ser. Lecture Notes in Computer Science, T. Okamoto, Ed., vol. 1976. Springer, 2000, pp. 73–89. [Online]. Available: https://doi.org/10.1007/3-540-44448-3_7
- [28] H. Devillez, O. Pereira, and T. Peters, "How to Verifiably Encrypt Many Bits for an Election?" in *Proceedings of Computer Security - ESORICS 2022, Part II*, ser. Lecture Notes in Computer Science, V. Atluri, R. D. Pietro, C. D. Jensen, and W. Meng, Eds., vol. 13555. Springer, 2022, pp. 653–671. [Online]. Available: https://doi.org/10.1007/978-3-031-17146-8_32
- [29] M. Abdalla, M. Bellare, and P. Rogaway, "The Oracle Diffie-Hellman Assumptions and an Analysis of DHIES," in *Proceedings of Topics in Cryptology - CT-RSA 2001*, ser. Lecture Notes in Computer Science, D. Naccache, Ed., vol. 2020. Springer, 2001, pp. 143–158. [Online]. Available: https://doi.org/10.1007/3-540-45353-9_12
- [30] M. Bellare and C. Namprepmpre, "Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm," in *Proceedings of Advances in Cryptology - ASIACRYPT 2000*, ser. Lecture Notes in Computer Science, T. Okamoto, Ed., vol. 1976. Springer, 2000, pp. 531–545. [Online]. Available: https://doi.org/10.1007/3-540-44448-3_41
- [31] J. H. An, Y. Dodis, and T. Rabin, "On the security of joint signature and encryption," in *Proceedings of Advances in Cryptology - EUROCRYPT 2002*, ser. Lecture Notes in Computer Science, L. R. Knudsen, Ed., vol. 2332. Springer, 2002, pp. 83–107. [Online]. Available: https://doi.org/10.1007/3-540-46035-7_6

- [32] D. Boneh, A. Partap, and L. Rotem, “Accountability for Misbehavior in Threshold Decryption via Threshold Traitor Tracing,” in *Proceedings of Advances in Cryptology - CRYPTO 2024, Part VII*, ser. Lecture Notes in Computer Science, L. Reyzin and D. Stebila, Eds., vol. 14926. Springer, 2024, pp. 317–351. [Online]. Available: https://doi.org/10.1007/978-3-031-68394-7_11
- [33] C. Baum, D. Cozzo, and N. P. Smart, “Using TopGear in Overdrive: A More Efficient ZKPoK for SPDZ,” in *Proceedings of Selected Areas in Cryptography - SAC 2019*, ser. Lecture Notes in Computer Science, K. G. Paterson and D. Stebila, Eds., vol. 11959. Springer, 2019, pp. 274–302. [Online]. Available: https://doi.org/10.1007/978-3-030-38471-5_12
- [34] I. Damgård, V. Pastro, N. P. Smart, and S. Zakarias, “Multiparty Computation from Somewhat Homomorphic Encryption,” in *Proceedings of Advances in Cryptology - CRYPTO 2012*, ser. Lecture Notes in Computer Science, R. Safavi-Naini and R. Canetti, Eds., vol. 7417. Springer, 2012, pp. 643–662. [Online]. Available: https://doi.org/10.1007/978-3-642-32009-5_38
- [35] H. Cui, K. Zhang, Y. Chen, Z. Liu, and Y. Yu, “MPC-in-Multi-Heads: A Multi-Prover Zero-Knowledge Proof System - (or: How to Jointly Prove Any NP Statements in ZK),” in *Proceedings of Computer Security - ESORICS 2021, Part II*, ser. Lecture Notes in Computer Science, E. Bertino, H. Schulmann, and M. Waidner, Eds., vol. 12973. Springer, 2021, pp. 332–351. [Online]. Available: https://doi.org/10.1007/978-3-030-88428-4_17
- [36] C. Asmuth and J. Bloom, “A modular approach to key safeguarding,” *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 208–210, 1983. [Online]. Available: <https://doi.org/10.1109/TIT.1983.1056651>
- [37] M. Barzu, F. L. Tiplea, and C. C. Dragan, “Compact sequences of co-primes and their applications to the security of CRT-based threshold schemes,” *Inf. Sci.*, vol. 240, pp. 161–172, 2013. [Online]. Available: <https://doi.org/10.1016/j.ins.2013.03.062>
- [38] T. H. Yuen, H. Cui, and X. Xie, “Compact Zero-Knowledge Proofs for Threshold ECDSA with Trustless Setup,” in *Proceedings of Public-Key Cryptography - PKC 2021, Part I*, ser. Lecture Notes in Computer Science, J. A. Garay, Ed., vol. 12710. Springer, 2021, pp. 481–511. [Online]. Available: https://doi.org/10.1007/978-3-030-75245-3_18
- [39] E. C. Crites, C. Komlo, and M. Maller, “Fully Adaptive Schnorr Threshold Signatures,” in *Proceedings of Advances in Cryptology - CRYPTO 2023, Part I*, ser. Lecture Notes in Computer Science, H. Handschuh and A. Lysyanskaya, Eds., vol. 14081. Springer, 2023, pp. 678–709. [Online]. Available: https://doi.org/10.1007/978-3-031-38557-5_22
- [40] F. Hao and P. Y. Ryan, *Real-world electronic voting: Design, analysis and deployment*. CRC Press, 2017, [https://realworlddevoting.com/\(visited 2025-11-01\)](https://realworlddevoting.com/(visited%2025-11-01)).
- [41] V. Cortier, D. Galindo, S. Glondou, and M. Izabachène, “Election Verifiability for Helios under Weaker Trust Assumptions,” in *Proceedings of Computer Security - ESORICS 2014, Part II*, ser. Lecture Notes in Computer Science, M. Kutylowski and J. Vaidya, Eds., vol. 8713. Springer, 2014, pp. 327–344. [Online]. Available: https://doi.org/10.1007/978-3-319-11212-1_19
- [42] H. Devillez, O. Pereira, and T. Peters, “Practical Traceable Receipt-Free Encryption,” in *Proceedings of Security and Cryptography for Networks - SCN 2024, Part I*, ser. Lecture Notes in Computer Science, C. Galdi and D. H. Phan, Eds., vol. 14973. Springer, 2024, pp. 367–387. [Online]. Available: https://doi.org/10.1007/978-3-031-71070-4_17
- [43] D. Bernhard and B. Warinschi, “Cryptographic Voting - A Gentle Introduction,” in *Foundations of Security Analysis and Design VII - FOSAD 2012/2013 Tutorial Lectures*, ser. Lecture Notes in Computer Science, A. Aldini, J. López, and F. Martinelli, Eds., vol. 8604. Springer, 2013, pp. 167–211. [Online]. Available: https://doi.org/10.1007/978-3-319-10082-1_7

A. Hardness Assumptions

DDH: Decisional Diffie Hellman: Let $(p, q, g) \leftarrow \$ \text{GSetup}()$ be a group. The adversary needs to distinguish between (g^x, g^z, g^{xy}) and (g^x, g^z, g^z) for $x, y, z \in \mathbb{Z}_q$.

B. Threshold PKE Definition

The threshold encryption public key encryption [32] $\text{TPKE}(t, n) = (\text{KGen}, \text{Enc}, \text{Dec}, \text{Combine})$ for $0 < t \leq n$

- $(pk, pkc, sk_1, \dots, pk_n) \leftarrow \text{KGen}(\lambda, t, n)$ takes the security parameter λ and produces a public key pk , a number of secret keys $sk_1, \dots, sk_n$, and a combiner public key pkc -secret key pair.
- $c \leftarrow \text{Enc}(pk, m)$ returns the ciphertext c that encrypts the message m using the public key pk . When we need the randomness we use $c \leftarrow \text{Enc}(pk, m; r)$ or $(c, r) \leftarrow \text{Enc}(pk, m)$.
- $d_i \leftarrow \text{Dec}(sk_i, c)$ returns a partial decryption d_i using parties i secret key sk_i .
- $m_\perp \leftarrow \text{Combine}(pkc, c, I, \{d_i\}_{i \in I})$ returns either a valid message m or an error symbol $\perp$. It combines the partial decryptions of I users.

C. Standard Instantiations for Zero-Knowledge

In Figure 6 we provide the pseudocode for the standard zero-knowledge instantiations for Schnorr proofs [17] (case $m=1$), Chaum-Pederson proofs [43], and proofs of multiple-exponentiations [7] (a generalized version of Schnorr).

D. Variants of EPKE: EPKE-v2.1.A and EPKE-v2.1.B

In Figure 7 we present the two variants of EPKE, where the initial MAC from EPKE-v2.0 has been replaced by the Schnorr proof SchZKP.

For completeness we introduce the setup phase of EG-v2.1 in Figure 8. The rest of EG-v2.1 follows same algorithms from EG-v2.0.

E. Cryptographic Details of the Extraction and Simulation in the Proof of Theorem 4

Proof. We prove the three properties. We work in the ROM for the hash function H .

Statement and relation: Let $pp = ((p, q, g), t, n, hkey)$ be public parameters and let I be the set of provers. Each prover $i \in I$ holds a witness $lesk_i \in \mathbb{Z}_q$ and corresponding public values $lepk_i = g^{lesk_i}$ and $A_i = A^{lesk_i}$ for a public base $A \in \langle g \rangle$.

Define

$$\begin{aligned} lepk &:= \prod_{i \in I} lepk_i \\ AS &:= \prod_{i \in I} A_i. \end{aligned}$$

The NP relation proved by MPZKP is

$$R(\{lesk_i\}_{i \in I}, st) = 1 \iff AS = A^{\sum_{i \in I} lesk_i}$$

ChPeZKP [16]: Chaum-Pederson zero-knowledge proof $R((v, a), pp, (A, B))$ for $v \in \{0, 1\}$ 1 : $((p, q, g), hkey, pk) \leftarrow pp$ 2 : $e \leftarrow ((A = g^a \wedge B = pk^{a+v}) \vee (A = g^a \wedge B = pk^{a+(1-v)}))$ 3 : return e $P((v, a) : pp, (A, B)[aux])$ 1 : $((p, q, g), hkey, pk) \leftarrow pp$ 2 : $u_0, u_1, c_0, c_1 \leftarrow \mathbb{Z}_q$ 3 : $(A_0, B_0) \leftarrow (g^{u_0}, pk^{u_0+vc_0})$ 4 : $(A_1, B_1) \leftarrow (g^{u_1}, pk^{u_1+(v-1)c_1})$ 5 : $c \leftarrow H(hkey; [aux], pk, A, B, A_0, B_0, A_1, B_1)$ 6 : $c_v \leftarrow (c - c_{1-v}) \bmod q$ 7 : $z_0 \leftarrow (u_0 - c_0 \cdot a) \bmod q$ 8 : $z_1 \leftarrow (u_1 - c_1 \cdot a) \bmod q$ 9 : $\pi \leftarrow (c_0, c_1, z_0, z_1)$ 10 : return π $V((pp, (A, B)[aux]), \pi)$ 1 : $((p, q, g), hkey, pk) \leftarrow pp$ 2 : $(c_0, c_1, z_0, z_1) \leftarrow \pi$ 3 : $(A_0, B_0) \leftarrow (g^{z_0} A^{c_0}, pk^{z_0} B^{c_0})$ 4 : $(A_1, B_1) \leftarrow (g^{z_1} A^{c_1}, pk^{z_1} B^{c_1})$ 5 : $c' \leftarrow H(hkey; [aux], pk, A, B, A_0, B_0, A_1, B_1)$ 6 : return $(c = c')$	ExpoZKP [7]: Exponential zero-knowledge proof $R((a_0, \dots, a_m) : pp, A_0, \dots, A_m)$ 1 : $((p, q, g), hkey) \leftarrow pp$ 2 : $\forall i \in \{0, \dots, m\}. e_i \leftarrow A_i = g^{a_i} \bmod p$ 3 : $e \leftarrow e_0 \wedge \dots \wedge e_m$ 4 : return e $P((a_0, \dots, a_m) : pp, A_0, \dots, A_m, [aux])$ 1 : $((p, q, g), hkey) \leftarrow pp$ 2 : $r_0, \dots, r_m \leftarrow \mathbb{Z}_q$ 3 : $\forall i \in \{0, \dots, m\}. h_i \leftarrow g^{r_i} \bmod p$ 4 : $c \leftarrow H_q(Hp; [aux], A_0, \dots, A_m, h_0, \dots, h_m)$ 5 : $\forall i \in \{0, \dots, m\}. z_i \leftarrow (r_i + c \cdot a_i) \bmod q$ 6 : $\pi \leftarrow (c, z_0, \dots, z_m)$ 7 : return π $V((pp, A_0, \dots, A_m, [aux]), \pi)$ 1 : $((p, q, g), hkey) \leftarrow pp$ 2 : $(c, z_0, \dots, z_m) \leftarrow \pi$ 3 : $\forall i \in \{0, \dots, m\}. h'_i \leftarrow (g^{z_i} / A_i^c) \bmod p$ 4 : $c' \leftarrow H_q(Hp; [aux], A_0, \dots, A_m, h'_0, \dots, h'_m)$ 5 : return $(c = c')$	SchZKP [17]: Schnorr proof $R(a : pp, A)$ 1 : $((p, q, g), Hp) \leftarrow pp$ 2 : $e \leftarrow A = g^a \bmod p$ 3 : return e $P_3(a : pp, A, [aux])$ 1 : $((p, q, g), hkey) \leftarrow pp$ 2 : $r \leftarrow \mathbb{Z}_q$ 3 : $h \leftarrow g^r \bmod p$ 4 : $c \leftarrow H(hkey; [aux], A, h)$ 5 : $z \leftarrow (r + c \cdot a) \bmod q$ 6 : $\pi \leftarrow (c, z)$ 7 : return π $V_3((pp, A, [aux]), \pi)$ 1 : $((p, q, g), hkey) \leftarrow pp$ 2 : $(c, z) \leftarrow \pi$ 3 : $h \leftarrow (g^z / A^c) \bmod p$ 4 : $c' \leftarrow H(hkey; [aux], A, h)$ 5 : return $(c = c')$
---	--	---

Fig. 6. [Appendix Figure] Instantiations for known Zero-Knowledge proof.

Completeness: Assume all provers are honest and the statement is true. Each prover samples $r_i \leftarrow \mathbb{Z}_q$ and publishes commitments

$$\begin{aligned} \forall i \in I. (U_i, V_i) &= (g^{r_i}, A^{r_i}) \\ c &= H(hkey; A, B, U, V, lepk, AS, [aux]) \\ \forall i \in I. z_i &:= r_i - c \cdot lesk_i \bmod q, \\ z &:= \sum_{i \in I} z_i \end{aligned}$$

Then,

$$g^z \cdot lepk^c = g^{\sum_i r_i - c \sum_i lesk_i} \cdot \left(\prod_i g^{lesk_i} \right)^c = \prod_i g^{r_i} = U,$$

and similarly

$$A^z \cdot AS^c = \prod_i A^{r_i} = V.$$

Hence the verifier reconstructs the same hash input and accepts. Therefore, MPZKP is complete.

Proof of knowledge: We show that MPZKP satisfies a proof-of-knowledge property via special soundness in the ROM.

Let $\mathcal{A}$ be a PPT. adversary that outputs a statement st and an accepting proof $\pi = (c, z)$. The extractor Ext runs $\mathcal{A}$ once to obtain (st, c, z) and rewinds $\mathcal{A}$ to the point just before the random oracle query that determines c . The extractor then reprograms the oracle to return an independent value $c' \neq c$, obtaining a second accepting proof (c', z') for the same statement.

Since both transcripts verify, we have

$$g^z \cdot lepk^c = g^{z'} \cdot lepk^{c'}.$$

Rearranging yields

$$g^{z-z'} = lepk^{c'-c} = g^{(\sum_{i \in I} lesk_i)(c'-c)}$$

Because g has order q , it follows that

$$\sum_{i \in I} lesk_i \equiv (z - z') \cdot (c' - c)^{-1} \bmod q.$$

Thus, the extractor recovers the aggregate witness $le sk = \sum_{i \in I} lesk_i$, which suffices to satisfy the relation $AS = A^{le sk}$. By the standard forking lemma argument in the ROM, if $\mathcal{A}$ succeeds with non-negligible probability, then Ext succeeds with non-negligible probability. Hence MPZKP is a proof of knowledge for the stated relation, and therefore sound.

Zero-knowledge: We construct a simulator Sim that produces accepting proofs without knowledge of any witness.

Given a statement st in the language, the simulator samples $c, z \leftarrow \mathbb{Z}_q$ uniformly at random and defines

$$\hat{U} := g^z \cdot lepk^c, \hat{V} := A^z \cdot AS^c$$

The simulator programs the random oracle so that

$$H(hkey; A, B, \hat{U}, \hat{V}, lepk, AS, [aux]) = c,$$

and outputs $\pi = (c, z)$ together with commitment-phase values whose product equals $(\hat{U}, \hat{V})$.

KGen ((p, q, g), hkey,) 1 : $sk \leftarrow \mathbb{Z}_q^*$; $pk \leftarrow g^{sk} \bmod p$ 2 : return (sk, pk)	Enc ((p, q, g), hkey, pk, m_0, m_1 , aux ₁ , aux ₂ , aux ₃) 1 : $r \leftarrow \mathbb{Z}_q$ 2 : $c_0 \leftarrow g^r \bmod p$ 3 : $\beta \leftarrow pk^r \bmod p$ 4 : $k \leftarrow H(hkey; aux_1, pk, c_0, \beta)$ 5 : $k_0 \leftarrow H(k; 0x01, aux_2)$ 6 : $k_1 \leftarrow H(k; 0x02, aux_2)$ 7 : $c_1 \leftarrow m_0 \oplus k_1$ 8 : $c_2 \leftarrow m_1 \oplus k_2$ 9 : $c_3 \leftarrow \text{SchZKP.P}(r : c_0, [aux_3, c_1, c_2])$ 10 : return ((c ₀ , c ₁ , c ₂ , c ₃), r)	Dec ((p, q, g), hkey, sk, (c ₀ , c ₁ , c ₂ , c ₃), aux ₁ , aux ₂ , aux ₃) 1 : $\beta \leftarrow c_0^{sk} \bmod p$ 2 : $k \leftarrow H(hkey; aux_1, pk, c_0, \beta)$ 3 : $k_0 \leftarrow H(k; 0x01, aux_2)$ 4 : $k_1 \leftarrow H(k; 0x02, aux_2)$ 5 : $m_0 \leftarrow c_1 \oplus k_1$ 6 : $m_1 \leftarrow c_2 \oplus k_2$ 7 : $e \leftarrow \text{SchZKP.V}(c_0, c_3, [aux_3, c_1, c_2])$ 8 : if $\neg e$ then (m ₀ , m ₁) $\leftarrow \perp$ 9 : return (m ₀ , m ₁)
--	---	--

KGen ((p, q, g), hkey) 1 : $sk \leftarrow \mathbb{Z}_q^*$; $pk \leftarrow g^{sk} \bmod p$ 2 : return (sk, pk)	Enc ((p, q, g), hkey, pk, m, aux ₁ , aux ₂ , aux ₃) 1 : $r \leftarrow \mathbb{Z}_q$ 2 : $c_0 \leftarrow g^r \bmod p$ 3 : $\beta \leftarrow pk^r \bmod p$ 4 : $k \leftarrow H(hkey; aux_1, c_0, \beta)$ 5 : $k_0 \leftarrow H(k; 0x01, aux_2)$ 6 : $c_1 \leftarrow m \oplus k_1$ 7 : $c_2 \leftarrow \text{SchZKP.P}(r : c_0, [aux_3, c_1])$ 8 : return ((c ₀ , c ₁ , c ₂), r)	Dec ((p, q, g), hkey, sk, (c ₀ , c ₁ , c ₂), aux ₁ , aux ₂ , aux ₃) 1 : $\beta \leftarrow c_0^{sk} \bmod p$ 2 : $k \leftarrow H(hkey; aux_1, pk, c_0, \beta)$ 3 : $k_0 \leftarrow H(k; 0x01, aux_2)$ 4 : $k_1 \leftarrow H(k; 0x02, aux_2)$ 5 : $m \leftarrow c_1 \oplus k_1$ 6 : $e \leftarrow \text{SchZKP.V}(c_0, c_2, [aux_3, c_1])$ 7 : if $\neg e$ then m $\leftarrow \perp$ 8 : return m
--	---	---

Fig. 7. [Appendix Figure] The EPKE-v2.1.B (top) and EPKE-v2.1.A (bottom), with highlighted differences from EPKE-v2.0 and DHIES [29].

In a real execution, the challenge c is uniformly distributed (as a random oracle output), and the response $z = \sum_i (r_i - c \cdot \text{les}_i)$ is uniform, conditioned on c . The simulator produces (c, z) with exactly the same distribution and ensures verification succeeds by construction. Therefore the simulated transcript is indistinguishable from a real one. MPZKP is zero-knowledge in the ROM. $\square$

F. Details of the IND-CPA Reduction in the Proof of Theorem 8

Proof. We prove the theorem by reduction to the IND-CPA security of DPKE.

Adversarial model: We consider a static adversary $\mathcal{A}$ that corrupts a fixed set Co of guardians with $|Co| < t$ before the protocol starts. The adversary controls all messages sent by corrupted guardians and learns their secret keys and secret shares. We assume no concurrent executions of the DKG protocol and that Schnorr proofs used during key generation are non-concurrent.

Goal: The adversary $\mathcal{A}$ participates in the threshold IND-CPA game: after observing a distributed key generation producing a public election key epk , it submits two plaintexts (m_0, m_1) and receives a challenge ciphertext encrypting m_β under epk , for a hidden bit $\beta \in \{0, 1\}$. Its goal is to guess β with non-negligible advantage.

We show how to construct an IND-CPA adversary $\mathcal{B}$ against DPKE such that any non-negligible advantage of $\mathcal{A}$ transfers to $\mathcal{B}$.

Reduction setup: Let $(pk^*, sk^*) \leftarrow \text{DPKE.KGen}$ be the challenge key pair given to $\mathcal{B}$ in the IND-CPA game.

The reduction $\mathcal{B}$ simulates the DKG-v2.0 protocol for $\mathcal{A}$ as follows:

- For each corrupted guardian $i \in Co$, $\mathcal{A}$ chooses (pk_i, sk_i) and provides a valid Schnorr proof of knowledge of sk_i .
- For each honest guardian $i \in Ho \setminus \{n\}$, $\mathcal{B}$ generates (pk_i, sk_i) honestly and simulates the corresponding proofs.
- For a distinguished honest guardian $n \in Ho$, $\mathcal{B}$ sets $pk_n := pk^*$ and does not know sk_n .

The simulated election public key is

$$epk := pk^* \cdot \prod_{i \in Ho \setminus \{n\}} pk_i \cdot \prod_{i \in Co} pk_i.$$

Simulation of secret sharing: In DKG-v2.0, each guardian distributes shares of its secret key using verifiable secret sharing. For all guardians except n , $\mathcal{B}$ simulates this process honestly.

For guardian n , $\mathcal{B}$ programs the distributed shares sent to corrupted guardians so that their public reconstructions are consistent with pk^* , without knowing sk^* . This is possible because shares are verified only via exponentiation and Schnorr proofs, and the reduction controls the randomness and proofs of honest parties.

Since at most $t-1$ guardians are corrupted, these simulated shares are information-theoretically hidden and indistinguishable from real ones.

Guardian G_i part in Setup	
<i>/ publish public keys of guardians $\forall i \in \text{GuL}$</i> <pre> 1 : $(pk_i, sk_i) \leftarrow \text{DPKE.KGen}(pp)$ 2 : $(\widehat{pk}_i, \widehat{sk}_i) \leftarrow \text{DPKE.KGen}(pp)$ 3 : $(\text{compk}_i, \text{comsk}_i) \leftarrow \text{DPKE.KGen}(pp)$ 4 : $\pi_i \leftarrow \text{ExpoZKP.P}((sk_i, \widehat{sk}_i, \text{comsk}_i) : pp, pk_i, \widehat{pk}_i, \text{compk}_i)$ 5 : $PL \leftarrow (i, pk_i, \widehat{pk}_i, \text{compk}_i, \pi_i)$ 6 : wait and collect $\forall j \neq i. (j, pk_j, \widehat{pk}_j, \text{compk}_j, \pi_j)$ 7 : $\forall j \neq i. \text{ if } \neg \text{ExpoZKP.V}(pp, pk_j, \widehat{pk}_j, \text{compk}_j, \pi_j) \text{ then}$ 8 : $\text{evidence} \leftarrow (j, pk_j, \widehat{pk}_j, \text{compk}_j, \pi_j)$ 9 : $\text{badL} \leftarrow \text{badL} \cup \{\text{Dispute}(i, j, \text{evidence})\}$ 10 : $\text{epk} = \prod_{j \in \text{GuL} \setminus \text{badL}} pk_j; \quad \widehat{\text{epk}} = \prod_{j \in \text{GuL} \setminus \text{badL}} \widehat{pk}_j$ </pre> <i>/ computer public key of election</i> <pre> 31 : $\text{epk} = \prod_{j \in \text{GuL} \setminus \text{badL}} pk_j; \quad \widehat{\text{epk}} = \prod_{j \in \text{GuL} \setminus \text{badL}} \widehat{pk}_j$ 32 : $H_E \leftarrow H(H_B; 0x14, \text{epk}, \widehat{\text{epk}})$ 33 : $\text{list} \leftarrow (\{pub_j j \in L\}, \{\widehat{pub}_j j \in L\}, \{\text{compk}_j j \in L\})$ 34 : $H_G \leftarrow H(H_E; 0x13, \text{epk}, \widehat{\text{epk}}, \text{list})$ 35 : return $((H_E, H_G, \text{epk}, \widehat{\text{epk}}), (\text{esk}_i, \widehat{\text{esk}}_i, \text{comsk}_i))$ </pre>	<i>/ Exchange shares of guardian public keys</i> <pre> 11 : $((sh_{i,1}, \dots, sh_{i,n}), \text{pub}_i) \leftarrow \text{VSS-v2.1.Share}(pp, sk_i, [0x10, i])$ 12 : $((\widehat{sh}_{i,1}, \dots, \widehat{sh}_{i,n}), \widehat{\text{pub}}_i) \leftarrow \text{VSS-v2.1.Share}(pp, \widehat{sk}_i, [0x10, i])$ 13 : $\text{caux} \leftarrow (\text{label}, 0x00, \text{context}, 0x0200)$ 14 : $\forall j \neq i. (c_{i,j}, r_{i,j}) \leftarrow \text{EPKE-v2.1.A.Enc}(pp, \text{compk}_j, sh_{i,j}, \widehat{sh}_{i,j}, [0x11, i, j], [\text{caux}])$ 15 : $PL \leftarrow (pk_i, \text{pub}_i, \widehat{\text{pub}}_i)$ 16 : individual send $c_{i,j}$ to Guardian G_j 17 : wait and collect $\forall j \neq i. (pk_j, pk_j, c_{j,i})$ 18 : $\forall j \neq i. (sh_{j,i}, \widehat{sh}_{j,i}) \leftarrow \text{EPKE-v2.1.A.Dec}(pp, sk_i, c_{j,i}, [0x11, j, i], [\text{caux}])$ 19 : $\forall j \neq i. \text{ if } \neg \text{VSS-v2.1.Vf}(pp, \text{pub}_j, sh_{j,i}, [0x10, j, i]) \text{ then}$ 20 : $\text{evidence}_1 \leftarrow (pk_i, pk_j, sh_{j,i}, c_{j,i})$ 21 : $\text{evidence}_2 \leftarrow (pk_i, pk_j, c_{j,i}, r_{j,i})$ 22 : $\text{badL} \leftarrow \text{badL} \cup \{\text{Dispute}(i, j, \text{evidence})\}$ 23 : $\forall j \neq i. \text{ if } \neg \text{VSS-v2.1.Vf}(pp, \widehat{\text{pub}}_j, \widehat{sh}_{j,i}, [0x10, j, i]) \text{ then}$ 24 : $\text{evidence}_1 \leftarrow (pk_i, pk_j, \widehat{sh}_{j,i}, c_{j,i})$ 25 : $\text{evidence}_2 \leftarrow (pk_i, pk_j, c_{j,i}, r_{j,i})$ 26 : $\text{badL} \leftarrow \text{badL} \cup \{\text{Dispute}(i, j, \text{evidence})\}$ 27 : $\text{esk}_i \leftarrow \sum_{j \in \text{GuL} \setminus \text{badL}} sh_{j,i} \bmod q \quad / \text{ shares of } sk_i \text{ are also used}$ 28 : $\text{epk}_i \leftarrow g^{\text{esk}_i} \bmod p$ 29 : $\widehat{\text{esk}}_i \leftarrow \sum_{j \in \text{GuL} \setminus \text{badL}} \widehat{sh}_{j,i} \bmod q$ 30 : $\widehat{\text{epk}}_i \leftarrow g^{\widehat{\text{esk}}_i} \bmod p$ </pre>

Fig. 8. [Appendix Figure] The Setup algorithm for EG-v2.1 with highlighted differences to EG-v2.0 (described in Figure 4 of main body). Changes are mostly due to running VSS.Share twice to build epk and $\widehat{\text{epk}}$, together with the v2.1 variant of the primitives VSS, EPKE, ExpoZKP.

Challenge phase: When $\mathcal{A}$ outputs challenge messages (m_0, m_1) , $\mathcal{B}$ forwards them to its IND-CPA challenger and receives a ciphertext

$$c^* = \text{DPKE.Enc}(pk^*, m_\beta).$$

$\mathcal{B}$ transforms c^* into a valid DPKE ciphertext under epk by homomorphically incorporating the known public keys of honest and corrupted guardians. This transformation preserves the distribution of encryptions under epk .

The resulting ciphertext is given to $\mathcal{A}$ as the challenge ciphertext.

Guess phase: Eventually, $\mathcal{A}$ outputs a guess β' . $\mathcal{B}$ outputs the same guess in the IND-CPA experiment.

Correctness of the reduction: If c^* encrypts m_β under pk^* , then the transformed ciphertext is distributed exactly as a DPKE encryption of m_β under epk . Therefore $\mathcal{A}$'s view is identical to that in a real threshold IND-CPA experiment.

If $\mathcal{A}$ distinguishes β with non-negligible advantage, then $\mathcal{B}$ distinguishes encryptions under pk^* with non-negligible advantage, contradicting the IND-CPA security of DPKE.

Conclusion: Since DPKE is IND-CPA secure under the DDH assumption, it follows that DPKE-with-DKG-v2.0 is threshold IND-CPA secure against static adversaries corrupting fewer than t guardians. $\square$